

# MLPy Workshop 7

March 4, 2026

## 1 Week 7 - Support Vector Machines

Darren Lim

### 1.0.1 Aims

The main concepts covered in this notebook are:

- understanding separable vs non-separable data
- implementing SVMs
- use of different kernels and parameter tuning in SVMs

1. Setup
2. SVC
3. Model assessment
4. Spam Data

This week, we will be exploring the basics of support vector machine models. We will be focusing on support vector machines for classification, which is provided by sklearn in the **SVC** model. For more details, please see <https://scikit-learn.org/stable/modules/svm.html>

The main class that we are using is [sklearn.svm.SVC](#)

As usual, during workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

## 2 Setup

### 2.1 Packages

Let's load the some of the packages needed for this workshop.

```
[1]: # Data libraries
import pandas as pd
import numpy as np

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC # SVM
from sklearn.preprocessing import StandardScaler # scaling features
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import GridSearchCV, KFold, StratifiedKFold
```

## 2.2 Helper Functions

This helper function plots the data and visualized the decision boundary and margin.

```
[2]: from sklearn.inspection import DecisionBoundaryDisplay
import sklearn.preprocessing

# Visualize the decision boundary and margin
# For D=2 inputs and binary classification
def plot_margin(model, X, y, figsize=(8,7)):

    fig, ax = plt.subplots(1,1,figsize=figsize)

    # Scatter plot of the inputs colored by class
    ax.scatter(X[:,0], X[:,1], c=y, s=30)

    # Show decision boundary
    DecisionBoundaryDisplay.from_estimator(
        model,
        X,
        plot_method="contour",
        colors="k",
        levels=[-1, 0, 1],
        linestyles=["--", "-", "--"],
        ax=ax,
    )

    # Highlight support vectors
    # If pipeline with StandardScaler, inverse transform the support vectors
    if (isinstance(model, sklearn.pipeline.Pipeline)):
```

```

        if (isinstance(model[0], sklearn.preprocessing.StandardScaler)):
            support_vectors = model[0].inverse_transform(model[-1].
↪support_vectors_)
        else:
            support_vectors = model[-1].support_vectors_
    else:
        support_vectors = model.support_vectors_
    ax.scatter(
        support_vectors[:, 0],
        support_vectors[:, 1],
        s=100,
        linewidth=1,
        facecolors="none",
        edgecolors="k",
    )
    plt.show()

```

### 3 Support Vector Classifier

The class `SVC` implements support vector classifiers and support vector machines. When creating an `SVC` object, various options are available, including:

- `C`: the inverse regularization parameter. NOTE: this defaults to `C=1` but should always be tuned.
- `kernel`: options include the **linear** kernel (`linear`), **polynomial** kernel (`poly`), **radial basis function** kernel (`rbf`), **sigmoid** kernel (`sigmoid`), or a user-defined kernel.
- `degree`: degree if using the **polynomial** kernel
- `gamma`: kernel coefficient for **rbf**, **polynomial**, or **sigmoid** kernels.
- `coef0`: additional coefficient term for the **polynomial** or **sigmoid** kernels.

After calling `.fit()`, the `SVC` object will have a number of attributes including:

- `support_vectors_`: containing the support vectors.

To predict the class labels from the fitted `SVC` model, we can call `.predict()`. And although SVMs only provide class labels and not the corresponding class probabilities, `SVC` provides a method to estimate the class probabilities by calling `.predict_proba()` (or `.predict_log_proba()` on the log scale) using Platt scaling. This uses cross-validation and makes the implementation slower, thus to turn on this option of estimating the class probabilities, you must first set `probability=True` when creating the `SVC` object. Also note that these probability estimates are unreliable on small datasets.

More details on kernels are available here: <https://scikit-learn.org/stable/modules/svm.html#svm-kernels>

#### 3.0.1 Difference between SVC and LinearSVC

Note the `sklearn` implements two linear support vector classification models `LinearSVC()` and `SVC(kernel='linear')`, which yield slightly different decision boundaries, due to the following differences:

- **LinearSVC** (based on LIBLINEAR) is **faster** than **SVC** (based on LIBSVM)
- **LinearSVC** minimizes the squared hinge loss while **SVC** minimizes the regular hinge loss.
- **LinearSVC** uses the One-vs-Rest scheme for multiclass classification while **SVC** uses the One-vs-One scheme for multiclass classification.
- **LinearSVC** does not provide some of the attributes of **SVC**, such as the support vectors.

For further details, see the documentations of the two classes

- **SVC** with `kernel=linear`: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- **LinearSVC**: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>

In the following, we will focus on the **SVC** class.

### 3.1 Linearly Separable Data

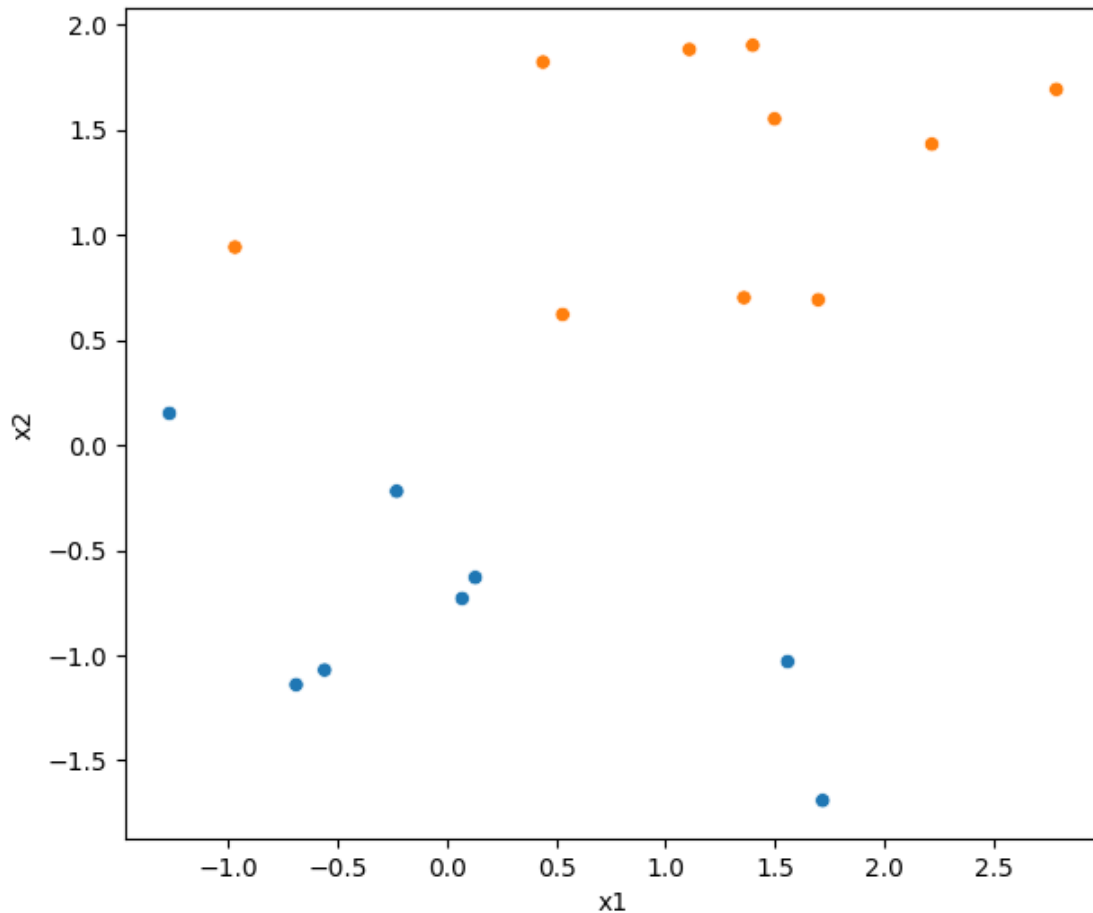
We will begin by examining various toy data sets to explore the basics of these models. For the first example, we will read in data from `ex1.csv`.

```
[3]: ex1 = pd.read_csv("ex1.csv")
     ex1.head()
```

```
[3]:      x1      x2  y
0 -0.56 -1.07  A
1 -0.23 -0.22  A
2  1.56 -1.03  A
3  0.07 -0.73  A
4  0.13 -0.63  A
```

Plotting the data below, we can see the that data is composed of two classes in two dimensions, and it is clear that these two classes are linearly separable.

```
[4]: plt.figure(figsize=(7,6))
     sns.scatterplot(x='x1', y='x2', hue='y', data=ex1, legend=False)
     plt.show()
```



Now, let separate the features and outcome and encode the outcome to a binary vector.

```
[5]: # Extracting the features and output and encoding y
X_ex1 = np.array(ex1.drop('y', axis=1))
y_ex1 = LabelEncoder().fit_transform(ex1.y)

print(X_ex1.shape)
print(y_ex1.shape)
```

```
(18, 2)
```

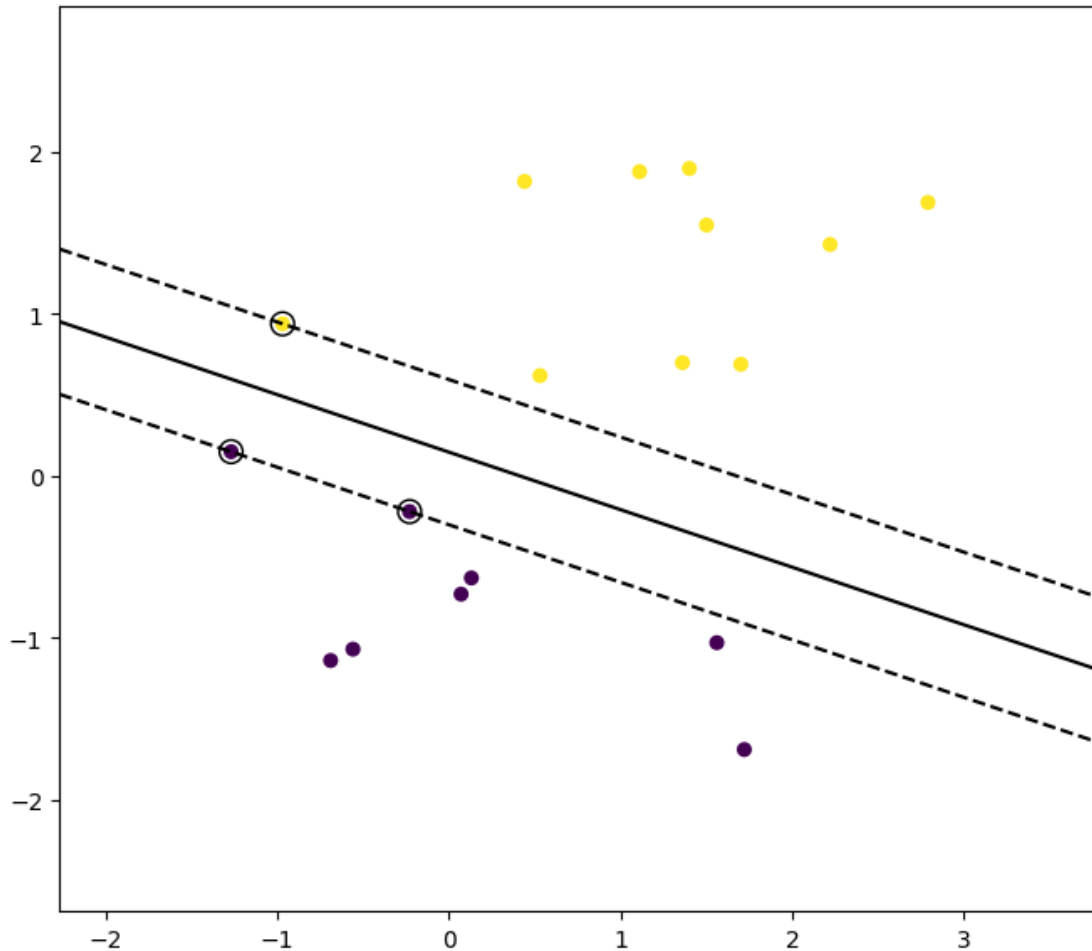
```
(18,)
```

### 3.1.1 Exercise 1 (CORE)

- Create and fit an SVC model using a **linear kernel** and **C=100**.
- Visualize the decision boundary, margin, and support vectors using the function `plot_margin` defined above. How many support vectors are there for each class? Are they on right side of the margin? hyperplane?

```
[6]: svc_lin_c100 = SVC(kernel = "linear", C = 100)
svc_lin_c100.fit(X_ex1, y_ex1)

plot_margin(svc_lin_c100, X_ex1, y_ex1)
```



Q1b)

- Class A has 2 support vectors, while class B has 1 support vector.
- All the support vectors are on the margin, and on the right side of the hyperplane.

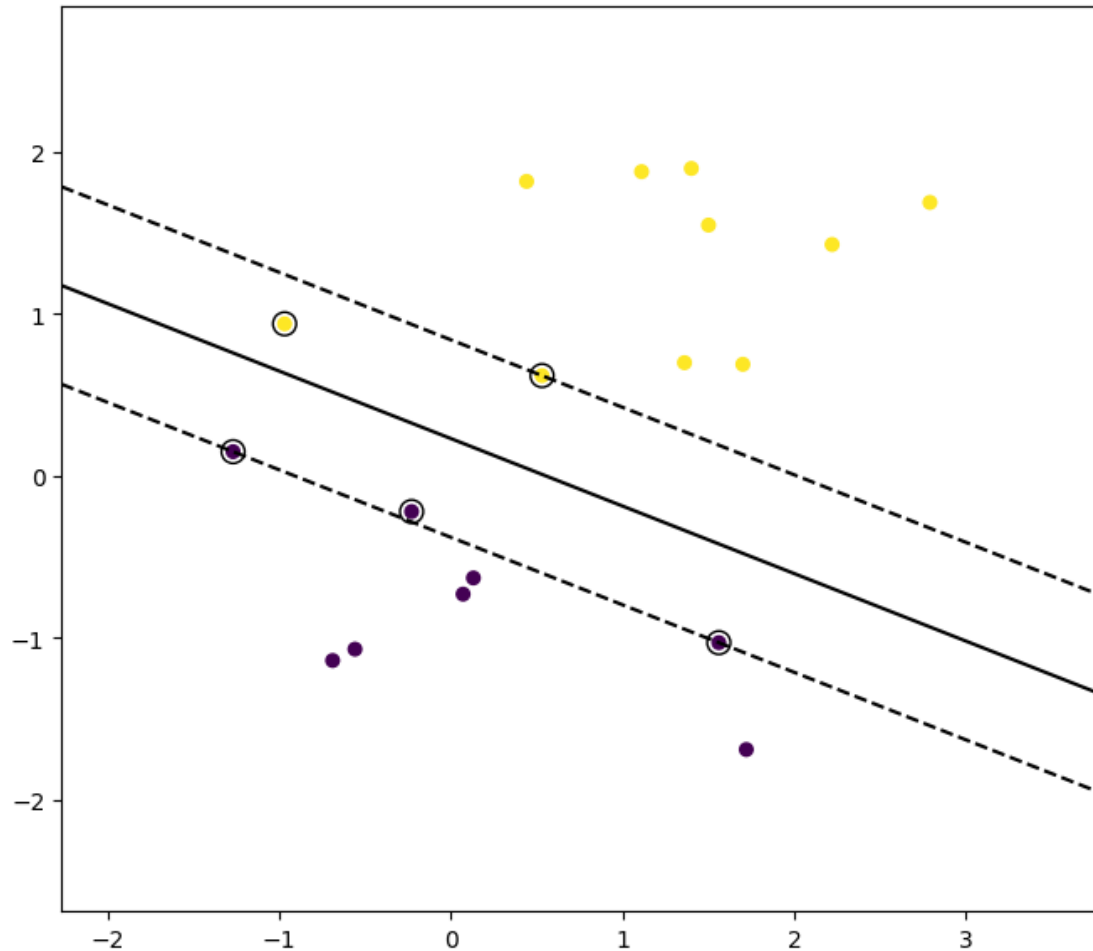
### 3.1.2 Exercise 2 (CORE)

Now, let's see how the results change with a small value of  $C$ .

- Create and fit an SVC model using a **linear kernel** and  $C=1$ .
- Visualize the decision boundary, margin, and support vectors. Now, how many support vectors are there for each class? Are they on right side of the margin? hyperplane? How has the margin changed?

```
[7]: svc_lin_c1 = SVC(kernel = "linear", C = 1)
svc_lin_c1.fit(X_ex1, y_ex1)

plot_margin(svc_lin_c1, X_ex1, y_ex1)
```



Q2b)

- Class A has 3 support vectors, while class B has 2 support vectors.
- All support vectors are on the right side of the hyperplane.
- 2 of class A support vectors are on the margin, while 1 of them is on the wrong side of the margin.
- 1 of class B support vectors is on the margin, while the other is on the wrong side of the margin.
- As  $C = 100$  decreased to  $C = 1$ , the margin has become wider, allowing for more training points to violate the margin.

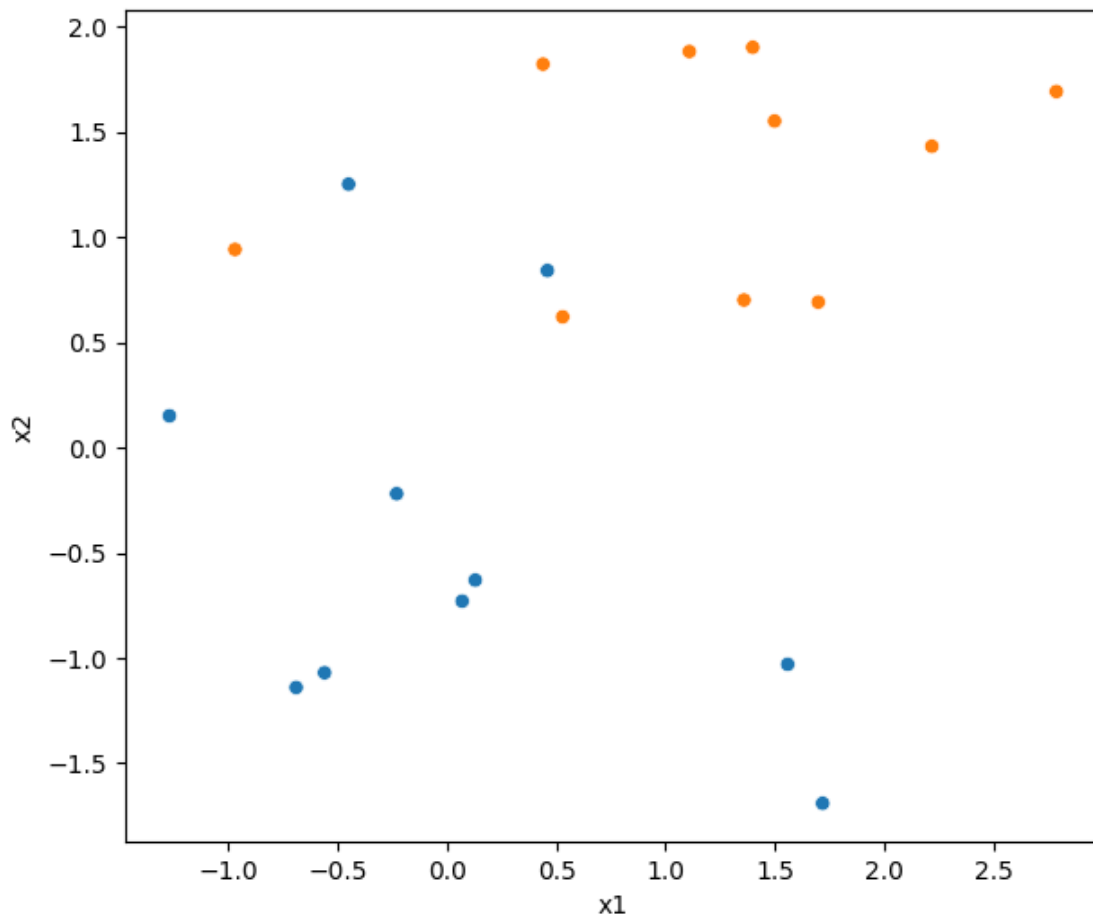
### 3.2 Non-Separable Data

Next, we complicate our previous example by adding two additional points from class A to our data, which result in data that are no longer linearly separable.

```
[8]: # Read in the data
ex2 = pd.read_csv("ex2.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex2, legend=False)
plt.show()

# Extracting the features and output and encoding y
X_ex2 = np.array(ex2.drop('y', axis=1))
y_ex2 = LabelEncoder().fit_transform(ex2.y)
```





### 3.2.1 Exercise 3 (CORE)

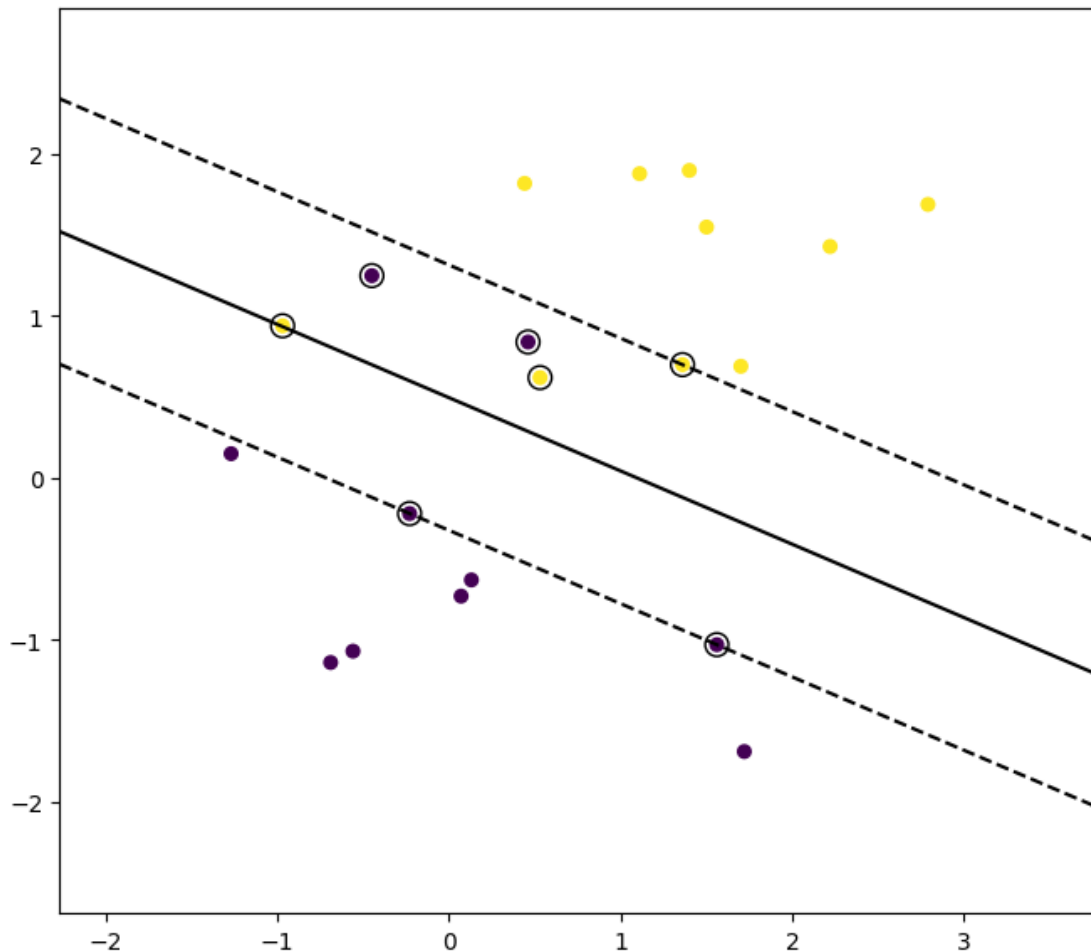
Fit the SVC model using the **linear kernel** and **same choices of  $C=100$  and  $C=1$** . How have the results changed? Comment on changes related to the margin and the number of support vectors for each class and their location relative to the margin and hyperplane.

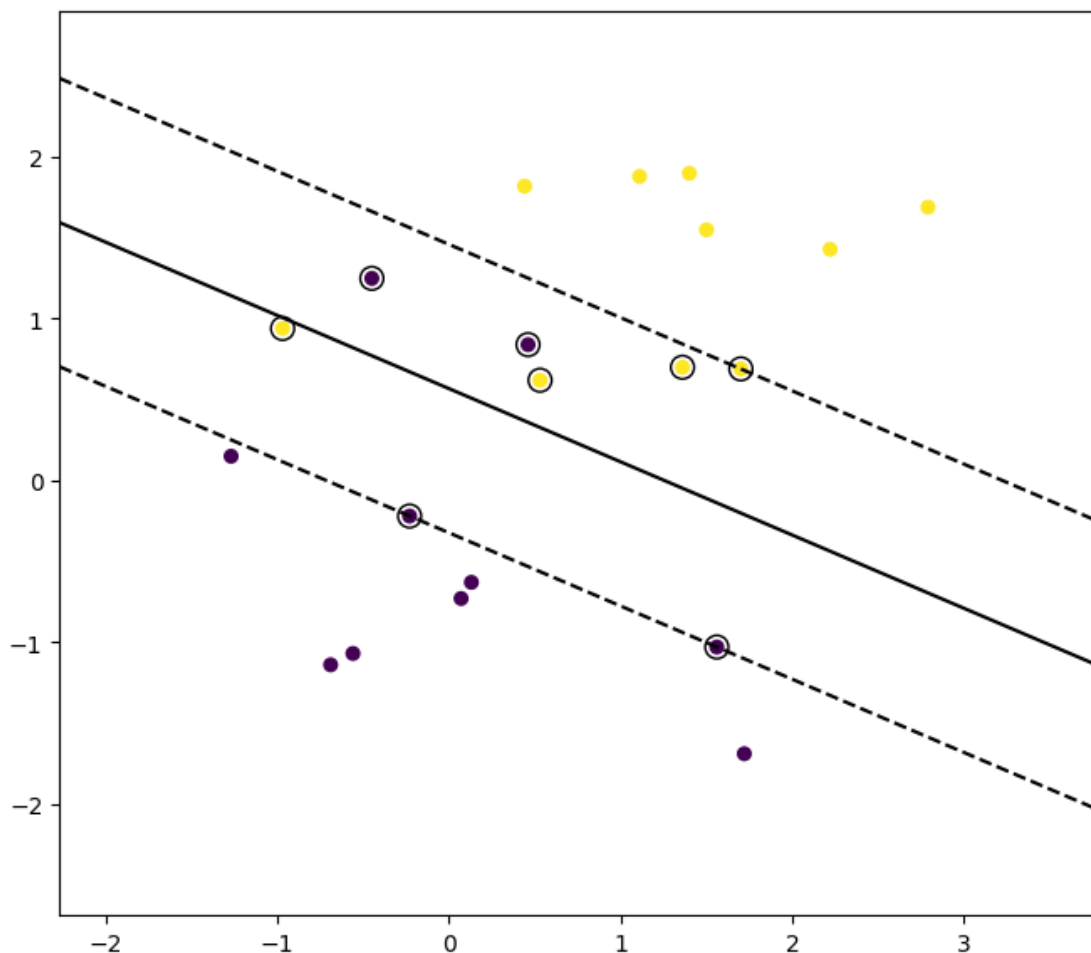
```
[9]: # C = 100 (small margin, more violations penalised)
svc_ex2_c100 = SVC(kernel = "linear", C = 100)
svc_ex2_c100.fit(X_ex2, y_ex2)

plot_margin(svc_ex2_c100, X_ex2, y_ex2)

# C = 1 (wider margin, more violations allowed)
svc_ex2_c1 = SVC(kernel = "linear", C = 1)
svc_ex2_c1.fit(X_ex2, y_ex2)

plot_margin(svc_ex2_c1, X_ex2, y_ex2)
```





Q3)

- In both models, class A has the same number of support vectors, that is, 4. In both models, 2 of class A support vectors lie on the margin, while the other 2 lie on the wrong side of the margin and the hyperplane.
- In the model with  $C = 100$ , class B has 3 support vectors: 1 lies on the margin, 1 lies in between the margin and the hyperplane, and 1 lies on the hyperplane. In the model with  $C = 1$ , class B has 4 support vectors: 1 lie on the margin, 2 lie in between the margin and the hyperplane, and 1 lies on the wrong side of the hyperplane.
- Compared to ex1, the data points in ex2 are not linearly separable, resulting in both models having to accept some margin and hyperplane violations.

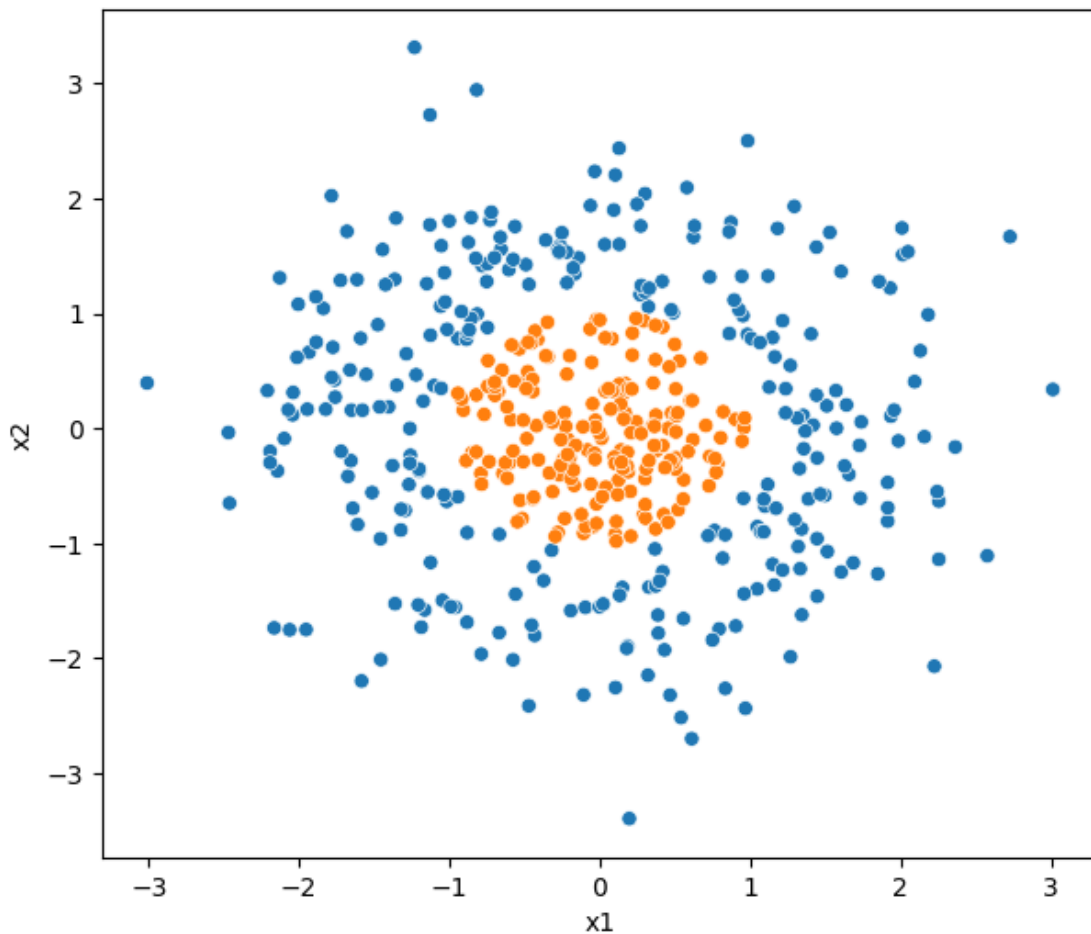
### 3.3 Nonlinear Separation

Next, we will look at a new data set that is not linearly separable, but can be perfectly separated by a nonlinear decision boundary. We will start by loading and visualizing the data.

```
[10]: # Load the third data set
ex3 = pd.read_csv("ex3.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex3, legend=False)
plt.show()

# Extracting the features and output and encoding y
X_ex3 = np.array(ex3.drop('y', axis=1))
y_ex3 = LabelEncoder().fit_transform(ex3.y)
```

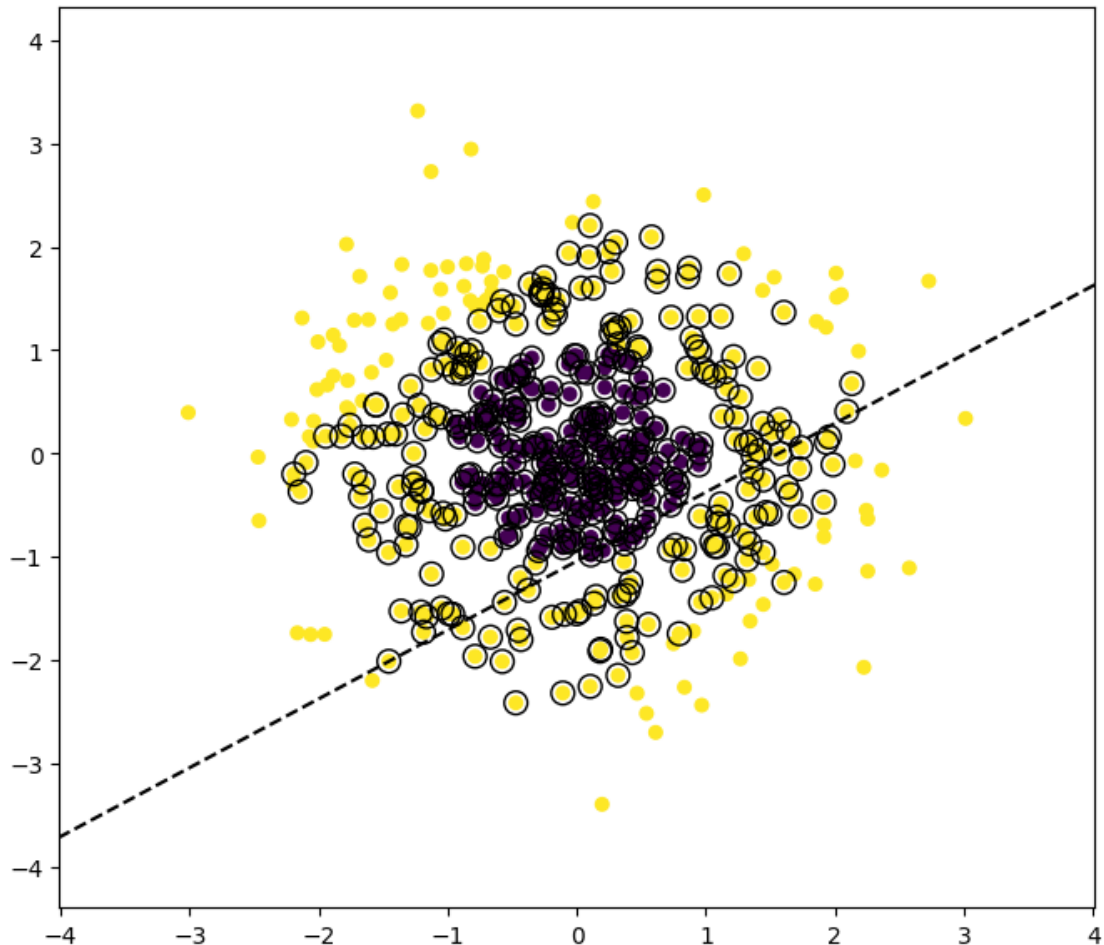


Let's start by trying to fit a linear SVC. As expected, the results are terrible and the number of support vectors is very large.

```
[11]: # Fit the SVC with a linear kernel on the new data
svm_lin = SVC(kernel='linear', C=100).fit(X_ex3, y_ex3)
```

```
# Visualize the decision boundary, margin and support vectors
plot_margin(svm_lin, X_ex3, y_ex3)

print('Number of support vectors for the linear model:', svm_lin.n_support_)
```



Number of support vectors for the linear model: [185 198]

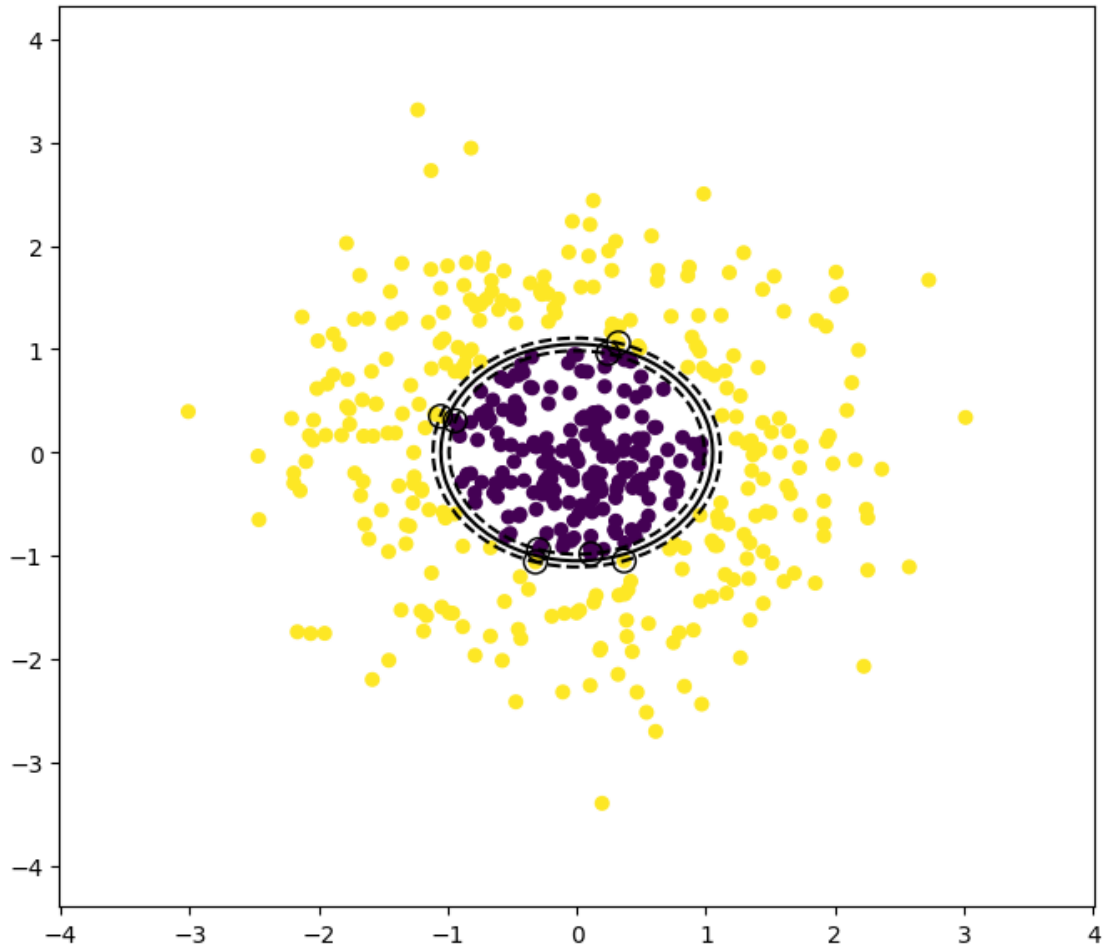
### 3.3.1 Exercise 4 (CORE)

- Fit an SVM with a **polynomial kernel with degree 2** and  $C=100$ .
- Visualize the margin and decision boundary and comment on the number of support vectors compared to the linear model.
- Compute and visualize the confusion matrix using `ConfusionMatrixDisplay.from_estimator`. Is the fitted SVM able to perfectly separate the classes?

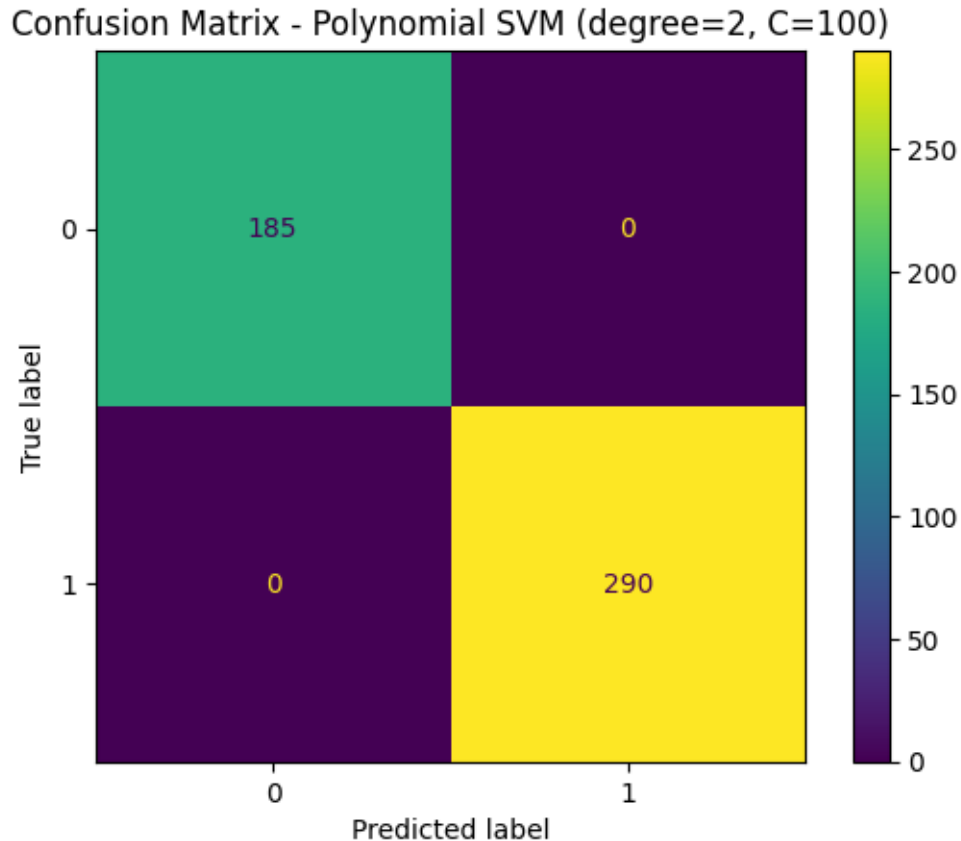
```
[12]: svm_poly = SVC(kernel = "poly", degree = 2, C = 100)
      svm_poly.fit(X_ex3, y_ex3)

      plot_margin(svm_poly, X_ex3, y_ex3)
      print("Support vectors per class (poly, deg=2):", svm_poly.n_support_)

      ConfusionMatrixDisplay.from_estimator(svm_poly, X_ex3, y_ex3)
      plt.title("Confusion Matrix - Polynomial SVM (degree=2, C=100)")
      plt.show()
```



Support vectors per class (poly, deg=2): [4 4]



Q4)

- Compared to the linear model, the polynomial model has significantly fewer support vectors.
- The degree-2 polynomial kernel has a quadratic circular decision boundary, accompanied by its circular margins.
- The confusion matrix shows 0 misclassifications, which suggests that the polynomial model performs decisively better and perfectly separates the 2 classes.

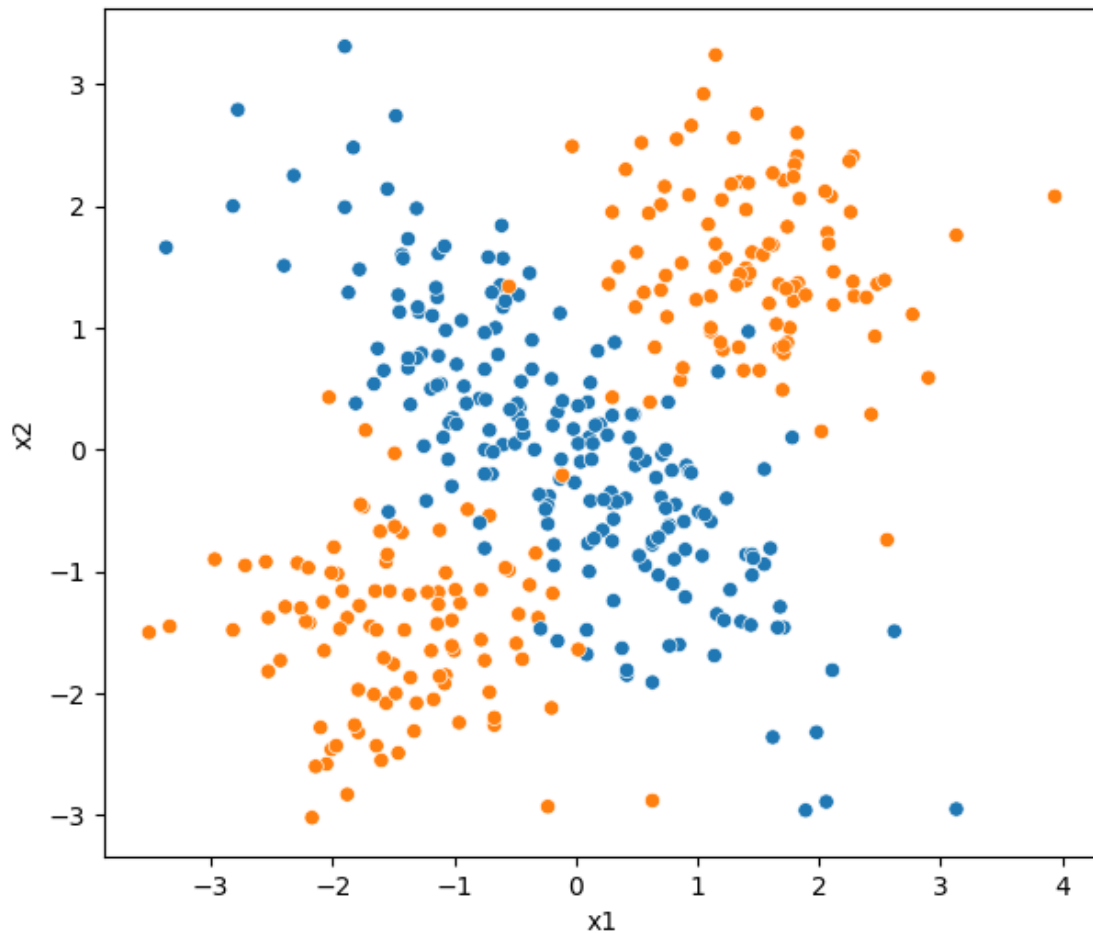
### 3.4 Kernels for SVMs and Parameter-Tuning

Next, we will consider an more complicated data, and explore using grid search to tune the model parameters and the effect of different kernels.

```
[13]: # Load the fourth data set
ex4 = pd.read_csv("ex4.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex4, legend=False)
plt.show()
```

```
# Extracting the features and output and encoding y
X_ex4 = np.array(ex4.drop('y', axis=1))
y_ex4 = LabelEncoder().fit_transform(ex4.y)
```



Let's start with the polynomial kernel that we saw in the previous exercise and use cross-validation to tune both the degree of polynomial and the penalty parameter.

```
[14]: # SVM with polynomial kernel
svm = SVC(kernel='poly', coef0=1)

# Grid search over C and the degree of the polynomial
degrees = [1,2,3,4]
C = np.linspace(0.1, 10, 100)
cv = GridSearchCV(
    svm,
    param_grid = {'C': C,
                  'degree': degrees},
```

```

    cv = KFold(5, shuffle = True, random_state = 0)
)

# Fit and tune the model
cv.fit(X_ex4, y_ex4)

# Get the best model parameters and the accuracy of the model
print("Params: ", cv.best_params_)
print("Avg Accuracy: ", cv.best_score_)

```

```

Params:  {'C': 3.1, 'degree': 2}
Avg Accuracy:  0.9425000000000001

```

### 3.4.1 Exercise 5 (CORE)

- Run the following code to plot the CV accuracy. Based on this plot, would you use the best parameter values printed above or choose different values? Why?
- Try changing the random state in Kfold to 11205 (or any other number of your choice). Do the best parameter values change? If so, why do you think this happens?
- For your selected parameters, fit the svm and plot the decision boundary and margin.

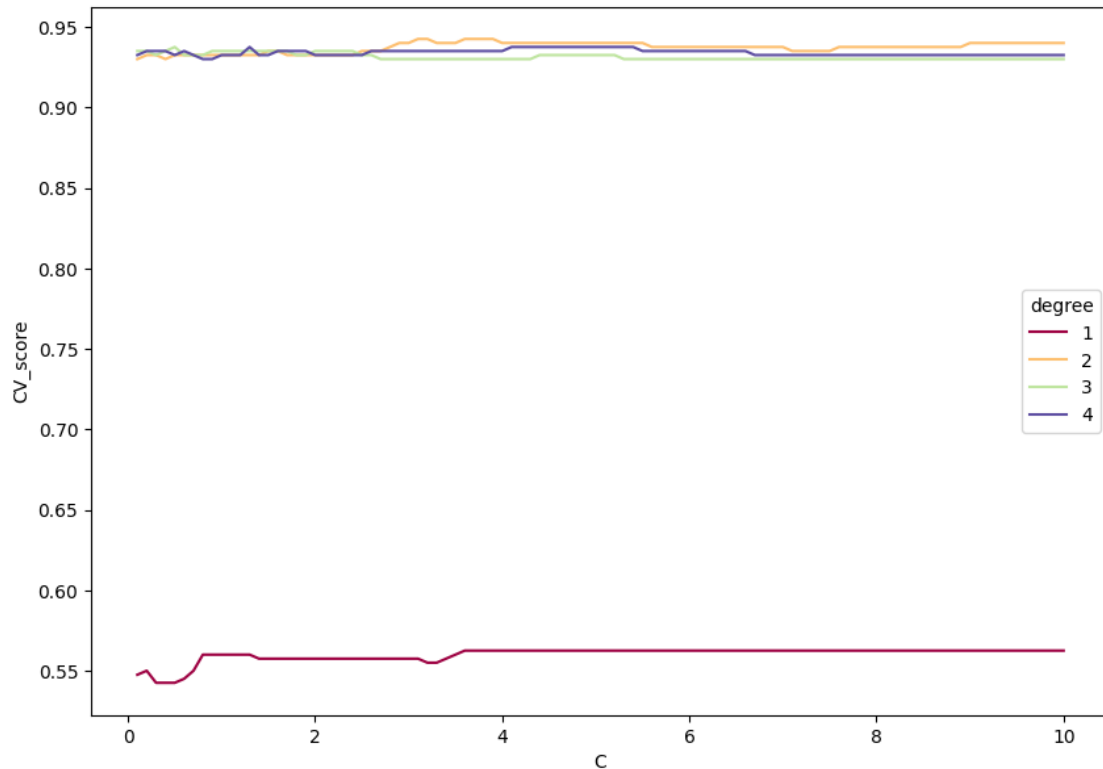
```

[15]: # Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv.cv_results_
                           ).filter(['param_C',
                                     ↪ 'param_degree', 'mean_test_score'])
                           ).rename(columns={'param_C': 'C',
                                     ↪ 'param_degree': 'degree', 'mean_test_score': 'CV_score'})

# Plot the CV scores
plt.figure(figsize=(10,7))
sns.lineplot(x='C', y='CV_score', data = cv_accuracy, hue = 'degree',
             ↪ palette="Spectral")
plt.show()

```





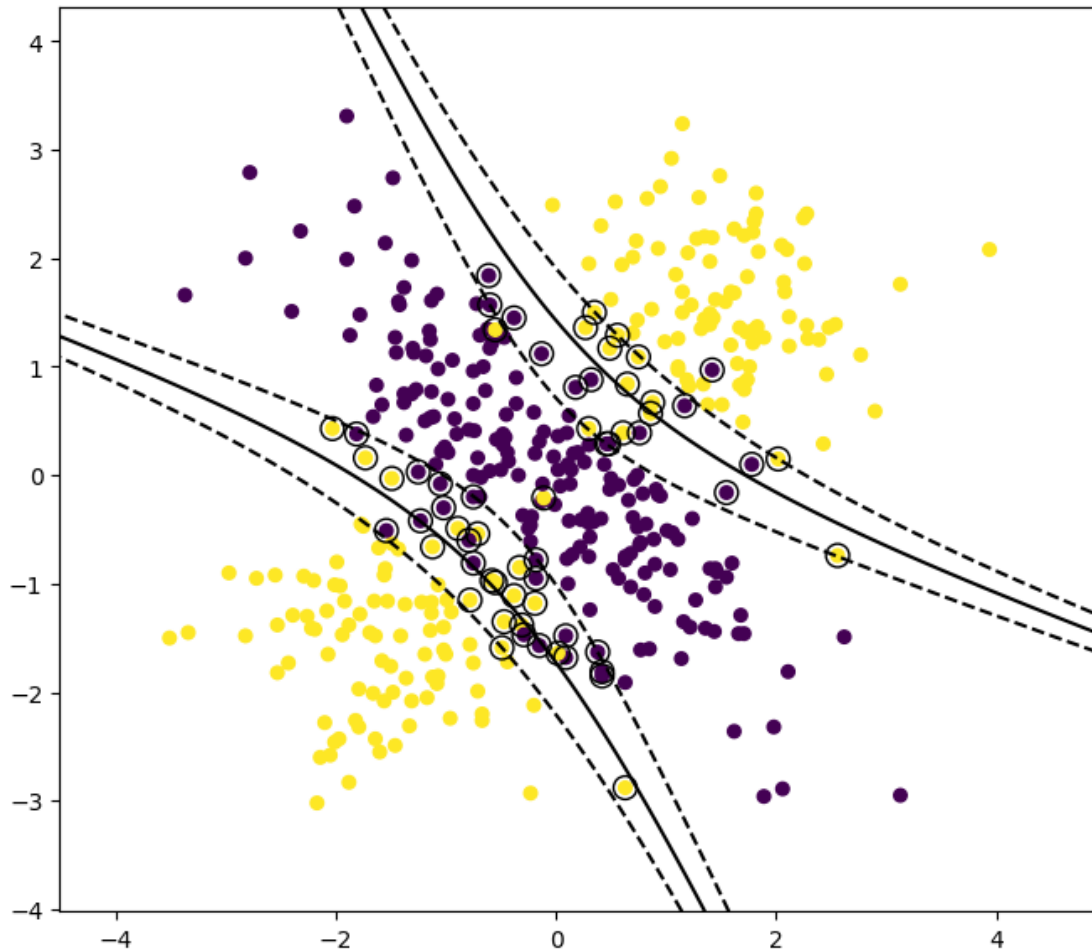
Q5)

- Based on the plot, only degree=1 performs significantly worse across all values of C, while degrees 2, 3, and 4 have similar, high performances. I would still use the chosen best parameters (C = 3.1, degree = 2), since degree=2 is the simplest model, yet performs similarly to degrees 3 and 4.
- At random\_state=11205, the best parameters change to (C = 0.4, degree = 3). This is to be expected, as changing the random\_state will change specific fold assignments and assignments to the validation set, resulting in different CV scores.

```
[16]: best_degree = 2
best_C = 3.1

svm_best = SVC(kernel = "poly", coef0 = 1, degree = best_degree, C = best_C)
svm_best.fit(X_ex4, y_ex4)

plot_margin(svm_best, X_ex4, y_ex4)
print(f"Chosen parameters: degree={best_degree}, C={best_C}")
```



Chosen parameters: degree=2, C=3.1

### 3.4.2 Exercise 6 (CORE)

- Repeat the grid search above with a polynomial kernel but set the coefficient term `coef0=0` (this is the default value). Redraw the plot of the CV accuracy. How have the results changed? Can you explain why?

```
[17]: svm_c0 = SVC(kernel = "poly", coef0 = 0)

degrees = [1, 2, 3, 4]
C = np.linspace(0.1, 10, 100)
cv_c0 = GridSearchCV(
    svm_c0,
    param_grid = {"C": C, "degree": degrees},
    cv = KFold(5, shuffle = True, random_state = 0)
)
cv_c0.fit(X_ex4, y_ex4)
```

```

print("Best params (coef0=0):", cv_c0.best_params_)
print("Best CV accuracy (coef0=0):", cv_c0.best_score_)

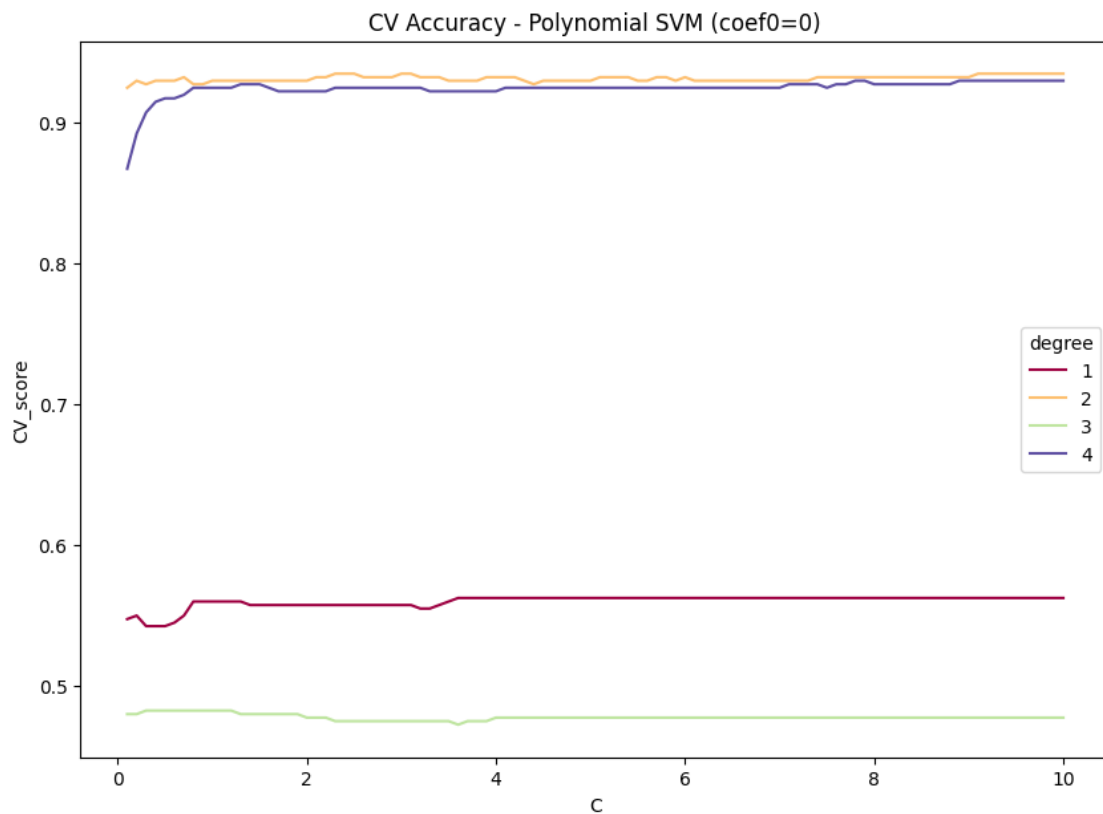
# Plot
cv_accuracy_c0 = pd.DataFrame(cv_c0.cv_results_
    .filter(["param_C", "param_degree", "mean_test_score"]
    ).rename(columns = {"param_C": "C", "param_degree": "degree",
        ↪ "mean_test_score": "CV_score"}))

plt.figure(figsize = (10, 7))
sns.lineplot(x = "C", y = "CV_score", data = cv_accuracy_c0, hue = "degree",
    ↪ palette = "Spectral")
plt.title("CV Accuracy - Polynomial SVM (coef0=0)")
plt.show()

```

Best params (coef0=0): {'C': 2.3000000000000003, 'degree': 2}

Best CV accuracy (coef0=0): 0.9349999999999999



Q6)

At  $\text{coef0}=0$ , the polynomial kernel becomes homogeneous, causing lower-degree terms including

the constant intercept to be absent and constraining the decision boundary to pass through the origin of the feature space. This can make the model less flexible for datasets that do not have their natural decision boundary passing through the origin, resulting in lower overall accuracy compared to `coef0=1`.

### 3.4.3 Exercise 7 (CORE)

Consider instead the RBF kernel.

- Use grid search to tune both the penalty parameter  $C$  and the inverse bandwidth parameter  $\gamma$ .
- Plot the decision boundary. How do the results compare to the SVM with a polynomial kernel?

```
[18]: svm_rbf = SVC(kernel = "rbf")

C_vals = np.linspace(0.1, 10, 20)
gamma_vals = np.logspace(-2, 1, 20)

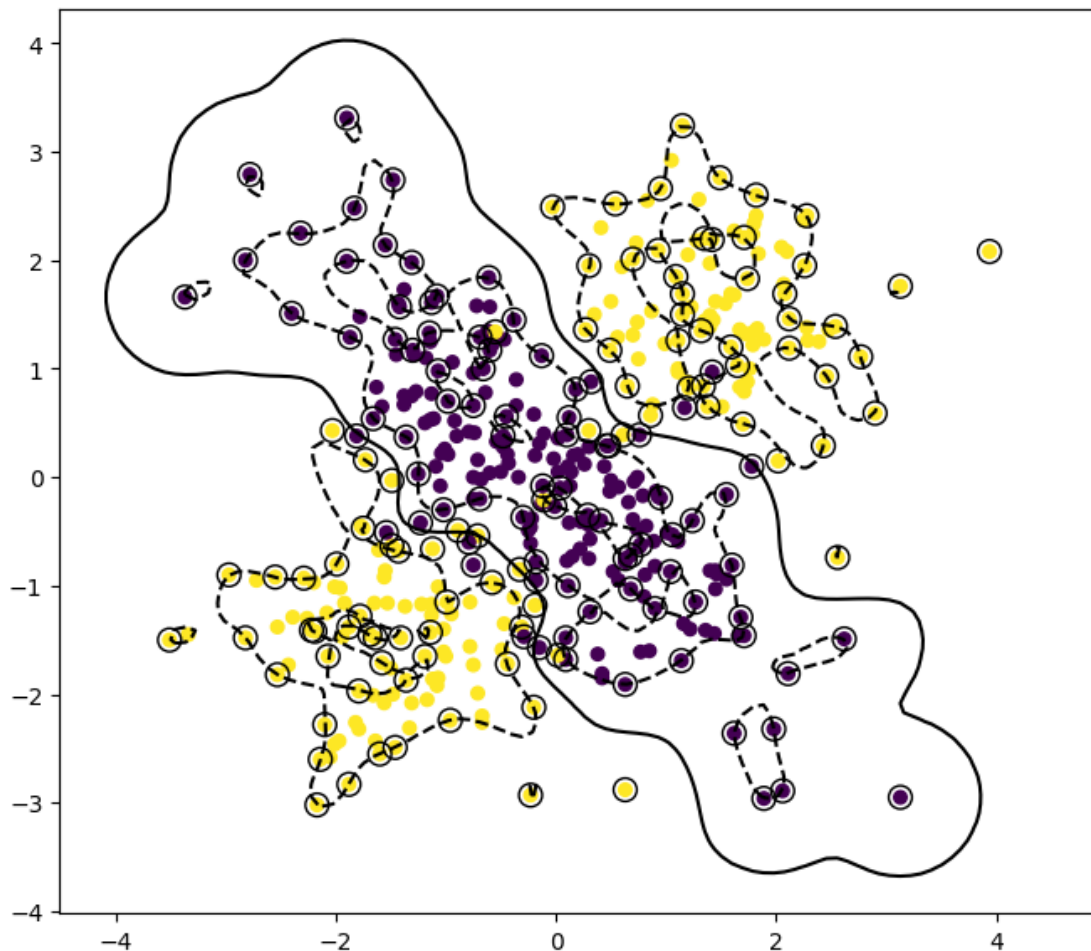
cv2 = GridSearchCV(
    svm_rbf,
    param_grid={"C": C_vals, "gamma": gamma_vals},
    cv=KFold(5, shuffle = True, random_state = 0)
)
cv2.fit(X_ex4, y_ex4)

print("Best params (RBF):", cv2.best_params_)
print("Best CV accuracy (RBF):", cv2.best_score_)

# Fit best RBF model and plot decision boundary
svm_rbf_best = cv2.best_estimator_
plot_margin(svm_rbf_best, X_ex4, y_ex4)
```

```
Best params (RBF): {'C': 1.6631578947368424, 'gamma': 4.832930238571752}
```

```
Best CV accuracy (RBF): 0.95
```



Q7)

The RBF kernel performs slightly better than, if not, similarly to the polynomial kernel. Its decision boundary tends to be smoother and more locally adaptive, identifying complex regions without the rigid polynomial structure.

### 3.5 Model Assessment

As, we learned last week, there are many metrics to consider beyond accuracy. For example, below we compute and print the classification report, summarizing the results for precision, recall, f1-score, and accuracy.

```
[19]: from sklearn.metrics import classification_report

# Classification report for the SVM with polynomial kernel
print(classification_report(y_ex4, cv.best_estimator_.predict(X_ex4)))

# Classification report for the SVM with RBF kernel
```

```
print(classification_report(y_ex4, cv2.best_estimator_.predict(X_ex4)))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.92      | 0.97   | 0.94     | 200     |
| 1            | 0.97      | 0.92   | 0.94     | 200     |
| accuracy     |           |        | 0.94     | 400     |
| macro avg    | 0.94      | 0.94   | 0.94     | 400     |
| weighted avg | 0.94      | 0.94   | 0.94     | 400     |

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 0.96   | 0.96     | 200     |
| 1            | 0.96      | 0.96   | 0.96     | 200     |
| accuracy     |           |        | 0.96     | 400     |
| macro avg    | 0.96      | 0.96   | 0.96     | 400     |
| weighted avg | 0.96      | 0.96   | 0.96     | 400     |

We also learned about other tools, such as the ROC curve, AUC, and precision-recall curve. However, since the SVMs are not model-based, **we only obtain hard label assignments when doing predictions**. To overcome this, one heuristic that can be used is **Platt scaling** to convert the SVM output to probabilities. However, these probabilities may not be well calibrated and may be inconsistent with the hard labels.

In the code below, we visualize the probabilities of class assignments across a grid of possible inputs for the tuned SVM polynomial model.

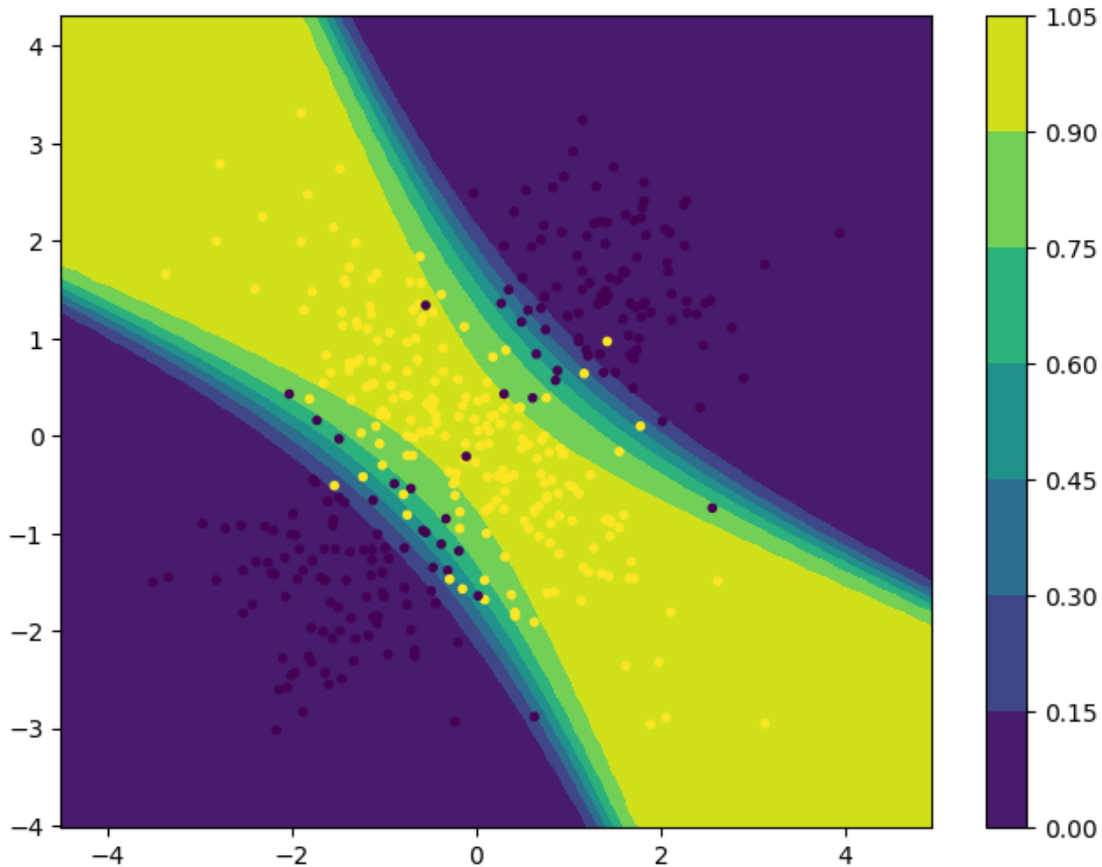
```
[20]: # Fit the polynomial model with optimal parameters and the option to compute
      ↪ the probabilities
svm_poly = SVC(kernel='poly', C=cv.best_params_["C"], degree=cv.
      ↪ best_params_["degree"], probability=True).fit(X_ex4, y_ex4)

# Create a grid of inputs for plotting
x1lim = [X_ex4[:,0].min(), X_ex4[:,0].max()]
x2lim = [X_ex4[:,1].min(), X_ex4[:,1].max()]

xx1 = np.linspace(x1lim[0]-1, x1lim[1]+1, 50)
xx2 = np.linspace(x2lim[0]-1, x2lim[1]+1, 50)
XX2, XX1 = np.meshgrid(xx2, xx1)

# Calculate the probability for each point in the grid
xx = np.c_[XX1.ravel(), XX2.ravel()]
xx.shape
P = svm_poly.predict_proba(xx)[:,0].reshape(XX1.shape)
```

```
plt.figure(figsize=(8,6))
plt.contourf(XX1, XX2, P)
plt.colorbar()
plt.scatter(x=X_ex4[:,0], y=X_ex4[:,1], c = 1-y_ex4, marker = '.')
```



Notice how the model is quite **confident in the predictions (probabilities close to 0 or 1)** in the corners of the input space, even where we don't have any data. In some cases, this may be undesirable, as we may not want to make such confident assessments in areas where we have little to no data.

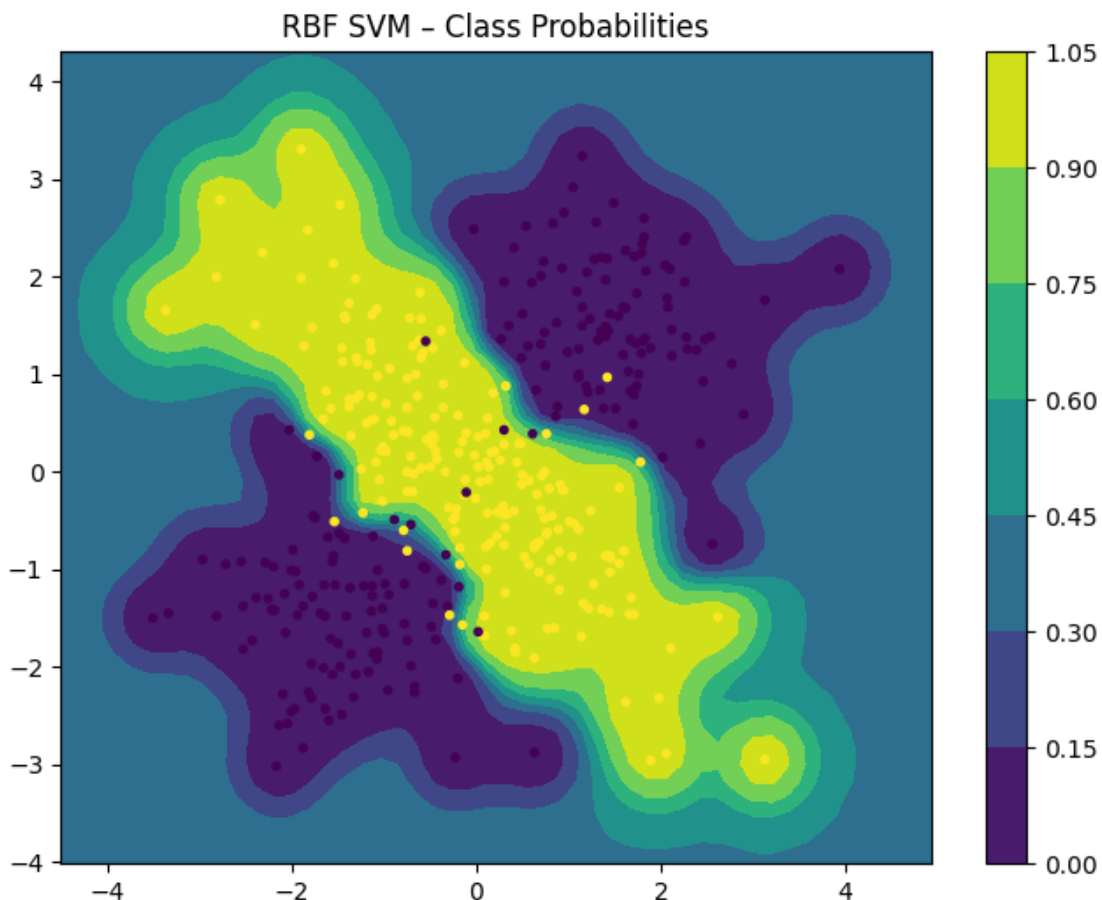
### 3.5.1 Exercise 8 (EXTRA)

- For the SVM with RBF kernel and optimal CV parameters, repeat the plot above to visualize the probabilities of class assignments.
- How do the results compare with the polynomial kernel? Does this impact your choice of kernel?

```
[21]: svm_rbf_prob = SVC(
    kernel='rbf',
    C=cv2.best_params_['C'],
    gamma=cv2.best_params_['gamma'],
    probability=True
).fit(X_ex4, y_ex4)

x1lim = [X_ex4[:,0].min(), X_ex4[:,0].max()]
x2lim = [X_ex4[:,1].min(), X_ex4[:,1].max()]
xx1 = np.linspace(x1lim[0]-1, x1lim[1]+1, 50)
xx2 = np.linspace(x2lim[0]-1, x2lim[1]+1, 50)
XX2, XX1 = np.meshgrid(xx2, xx1)
xx = np.c_[XX1.ravel(), XX2.ravel()]
P_rbf = svm_rbf_prob.predict_proba(xx)[: ,0].reshape(XX1.shape)

plt.figure(figsize=(8,6))
plt.contourf(XX1, XX2, P_rbf)
plt.colorbar()
plt.scatter(x=X_ex4[:,0], y=X_ex4[:,1], c=1-y_ex4, marker='.')
plt.title('RBF SVM - Class Probabilities')
plt.show()
```





Q8)

Unlike the polynomial SVM which is confident everywhere including data-sparse corners, the RBF SVM's probability surface tends to 0.5 in regions far from any training data. This is because the RBF kernel has local support: predictions in areas with no nearby training points receive little contribution from the kernel, yielding more uncertain estimates. This more conservative behaviour in unseen regions may be preferable in applications where over-confident extrapolation is dangerous.

## 4 Spam Data

Let's consider the spam data available on [kaggle](#). The dataset includes thousands of messages that have been marked as `spam` or `ham`. We will start by loading and exploring the data.

```
[22]: # Load the data
df = pd.read_csv("spam.csv", encoding='latin-1')

# Check information about the dataset
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0    v1              5572 non-null   object
1    v2              5572 non-null   object
2    Unnamed: 2      50 non-null     object
3    Unnamed: 3      12 non-null     object
4    Unnamed: 4       6 non-null     object
dtypes: object(5)
memory usage: 217.8+ KB
None
```

Let's do some basic data cleaning.

```
[23]: # Drop unnecessary columns
columns_to_drop = ["Unnamed: 2", "Unnamed: 3", "Unnamed: 4"]
df.drop(columns=columns_to_drop, inplace=True)

# Rename columns for better readability
new_column_names = {"v1": "Category", "v2": "Message"}
df.rename(columns=new_column_names, inplace=True)

print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
```

```
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Category    5572 non-null   object
1   Message      5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
None
```

Looking at the first few rows of the data (below), we see that we will need to do some feature engineering to covert the text messages into numeric features.

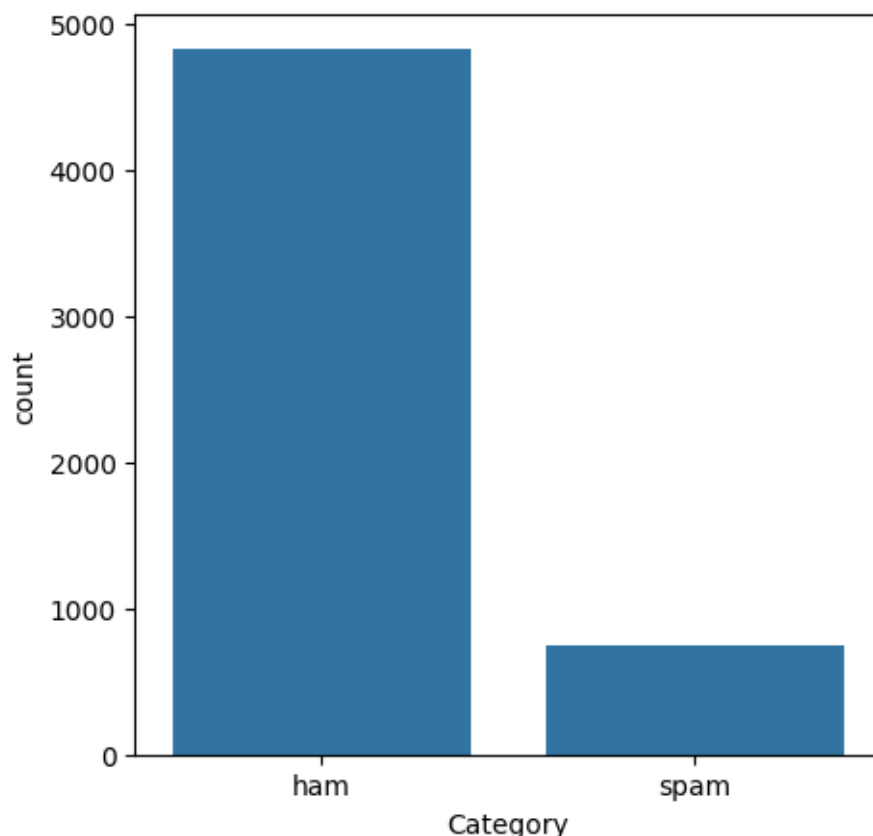
```
[24]: df.head()
```

```
[24]:   Category      Message
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
```

Let's start by splitting our data into features and targets. Notice that the dataset contains, as expected, less spam emails than ham.

```
[25]: # Split the dataset into features (X) and target (y)
X = df["Message"]
y = df["Category"]

plt.figure(figsize=(5,5))
sns.countplot(x=y)
plt.show()
```



We are going to use a simple **bag of words** transformation to convert our message to numeric features through sklearn's transformer: [CountVectorizer](#).

We will incorporate it into our pipeline later, but you can run the code below to get a sense of how it transforms the messages.

For simplicity, we set: - `binary=True` which returns binary features indicating the presence/absence of the words in the vocabulary - `max_features=50` and `stop_words="english"` which automatically extracts the vocabulary as the 50 most frequent words among the messages, excluding common english words like "and", "the", "him".

We can print the attribute `.vocabulary_` of our fitted transformer to see the words of our vocabulary.

```
[26]: # Create count vectorizer for bag of words feature extraction
from sklearn.feature_extraction.text import CountVectorizer

fe = CountVectorizer(max_features=50, stop_words="english", binary=True).fit(X)
X_ = fe.transform(X)

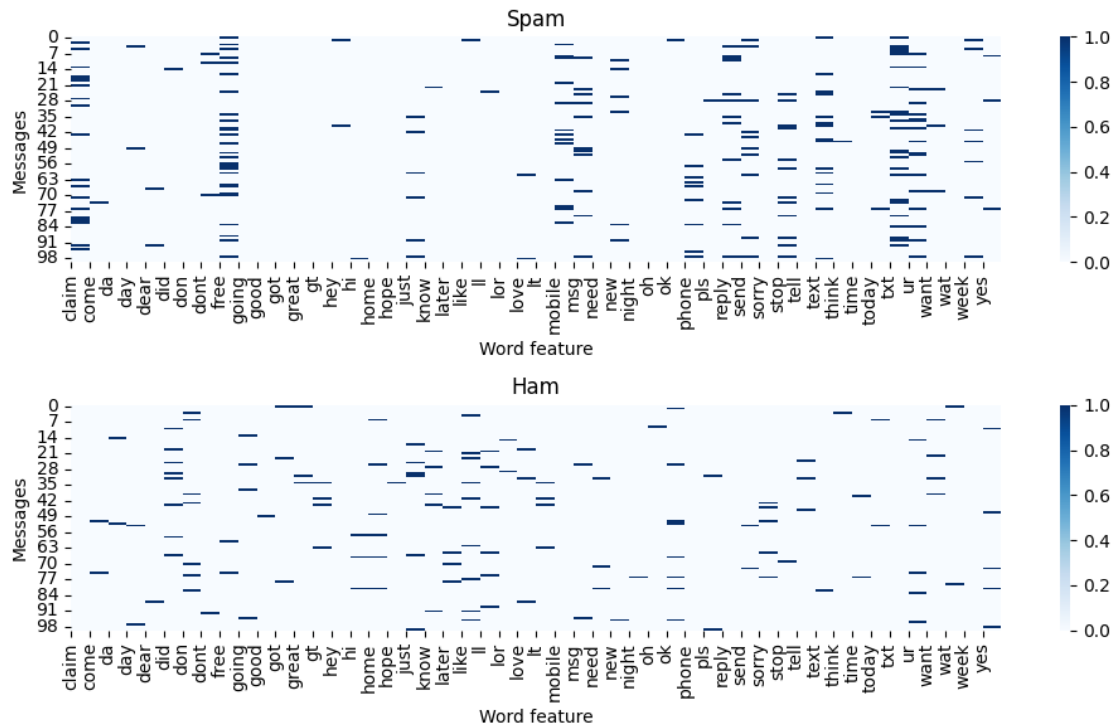
# print vocabulary
fe.vocabulary_
```

```
[26]: {'great': 12,  
      'got': 11,  
      'wat': 47,  
      'ok': 32,  
      'free': 8,  
      'text': 40,  
      'txt': 44,  
      'don': 6,  
      'think': 41,  
      'hey': 14,  
      'week': 48,  
      'like': 21,  
      'send': 36,  
      'claim': 0,  
      'mobile': 26,  
      'home': 16,  
      'want': 46,  
      'today': 43,  
      'day': 3,  
      'reply': 35,  
      'oh': 31,  
      'yes': 49,  
      'did': 5,  
      'dont': 7,  
      'ur': 45,  
      'going': 9,  
      'da': 2,  
      'lor': 23,  
      'just': 18,  
      'love': 24,  
      'll': 22,  
      'know': 19,  
      'tell': 39,  
      'msg': 27,  
      'pls': 34,  
      'need': 28,  
      'hope': 17,  
      'lt': 25,  
      'gt': 13,  
      'time': 42,  
      'sorry': 37,  
      'new': 29,  
      'later': 20,  
      'good': 10,  
      'come': 1,  
      'hi': 15,  
      'stop': 38,
```

```
'night': 30,  
'dear': 4,  
'phone': 33}
```

To visualize the transformed data, let's also draw a heatmap of first 100 messages for spam and ham messages.

```
[27]: # Draw a heatmap of the first 100 messages  
feature_names = fe.get_feature_names_out()  
X_pd_spam = pd.DataFrame(  
    X_.toarray()[y=='spam',:][0:100,:],  
    columns=feature_names  
)  
X_pd_ham = pd.DataFrame(  
    X_.toarray()[y=='ham',:][0:100,:],  
    columns=feature_names  
)  
  
fig, ax = plt.subplots(2,1,figsize=(10,6))  
sns.heatmap(X_pd_spam, cmap='Blues', vmin=0, vmax=1, ax=ax[0])  
ax[0].set_xlabel('Word feature')  
ax[0].set_ylabel('Messages')  
ax[0].set_title('Spam')  
ax[0].set_xticks(np.arange(len(feature_names)), labels = feature_names)  
sns.heatmap(X_pd_ham, cmap='Blues', vmin=0, vmax=1, ax=ax[1])  
ax[1].set_xlabel('Word feature')  
ax[1].set_ylabel('Messages')  
ax[1].set_title('Ham')  
ax[1].set_xticks(np.arange(len(feature_names)), labels = feature_names)  
plt.yticks(rotation=0)  
plt.tight_layout()  
plt.show()
```



Next, we split the data into a train and test set. Due to the class imbalance in the data, we use the option `stratify=y` to maintain the class proportion in the test set.

```
[28]: from sklearn.model_selection import train_test_split

y = LabelEncoder().fit_transform(y)

# Stratify split
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True,
    ↪stratify=y,

                                                    test_size = 0.1,
    ↪random_state=1112)
print(X_train.shape)
print('Proportion of spam in training', np.mean(y_train))
print('Proportion of spam in test', np.mean(y_test))
```

```
(5014,)
Proportion of spam in training 0.1340247307538891
Proportion of spam in test 0.13440860215053763
```

#### 4.0.1 Exercise 9 (CORE)

- Run the following code to fit an SVC and tune the penalty parameter `C`.
- Plot the accuracy, recall, precision, and f1 score as function of `C`. What value of `C` would you

choose and why?

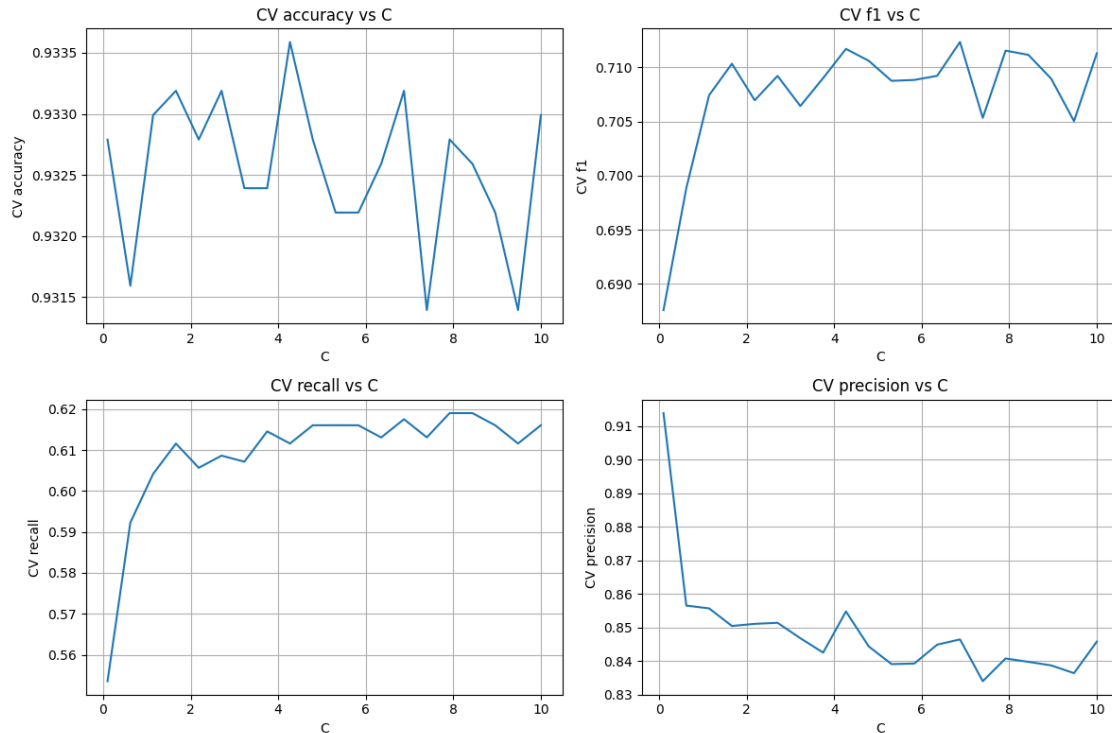
```
[29]: # SVM with linear kernel
svm_lin = make_pipeline(
    CountVectorizer(max_features=50, stop_words="english", binary=True),
    SVC(kernel='linear')
)

# Grid search over C
C = np.linspace(0.1, 10, 20)
cv_lin = GridSearchCV(
    svm_lin,
    param_grid = {'svc__C': C},
    cv = StratifiedKFold(5, shuffle = True, random_state = 0),
    scoring = ["accuracy", "f1", "recall", "precision"],
    refit='accuracy' #refit based on accuracy
)

# Fit and tune the model
cv_lin.fit(X_train, y_train)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv_lin.cv_results_
                           ).
    ↪filter(['param_svc__C', 'mean_test_accuracy', 'mean_test_f1',
    ↪'mean_test_recall', 'mean_test_precision']
          ).rename(columns={'param_svc__C':
    ↪'C', 'mean_test_accuracy': 'CV accuracy', 'mean_test_f1': 'CV f1',
    ↪'mean_test_recall': 'CV recall', 'mean_test_precision': 'CV precision'})

[30]: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
metrics = ['CV accuracy', 'CV f1', 'CV recall', 'CV precision']
for ax, metric in zip(axes.ravel(), metrics):
    ax.plot(cv_accuracy['C'], cv_accuracy[metric])
    ax.set_xlabel('C')
    ax.set_ylabel(metric)
    ax.set_title(metric + ' vs C')
    ax.grid(True)
plt.tight_layout()
plt.show()
```



Q9b)

The plots show that F1 increases as  $C$  increases, while accuracy fluctuates. Recall and precision trade off as  $C$  increases. Given the imbalanced data, I would choose the value of  $C=1.5$  that gives the best F1 score, while achieving a good balance between recall and precision.

#### 4.0.2 Exercise 10 (CORE)

- Refit the model with your chosen value of  $C$ . You should call your model `svm_lin` or change the name in code for part b.
- Run the code the below to visualize the coefficients and the support vectors for each class. Briefly comment on words/features that are relevant for distinguishing between the classes.

```
[31]: # Part a: code for your answer here
# Name your model svm_lin or change the name in the code below
chosen_C = 1.0

svm_lin = make_pipeline(
    CountVectorizer(max_features = 50, stop_words = "english", binary = True),
    SVC(kernel = "linear", C = chosen_C)
)
svm_lin.fit(X_train, y_train)
```

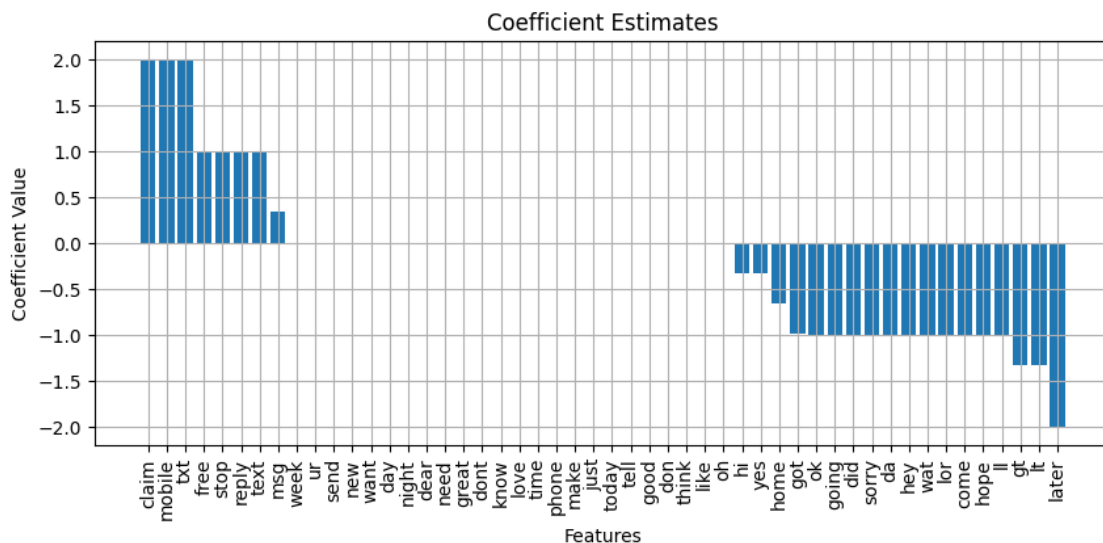


```
[31]: Pipeline(steps=[('countvectorizer',
                        CountVectorizer(binary=True, max_features=50,
                                        stop_words='english')),
                      ('svc', SVC(kernel='linear'))])
```

```
[32]: # Plot the coefficients of the linear SVC
feature_names = svm_lin[-1].get_feature_names_out()
w = svm_lin[-1].coef_.toarray().reshape(-1)

df = pd.DataFrame({'feature': feature_names, 'coef': w}).sort_values ("coef",
↪ascending=False)

plt.figure(figsize=(10,4))
plt.bar(df['feature'], df['coef'])
plt.title('Coefficient Estimates')
plt.xlabel('Features')
plt.ylabel('Coefficient Value')
plt.xticks(rotation=90)
plt.grid()
plt.show()
```



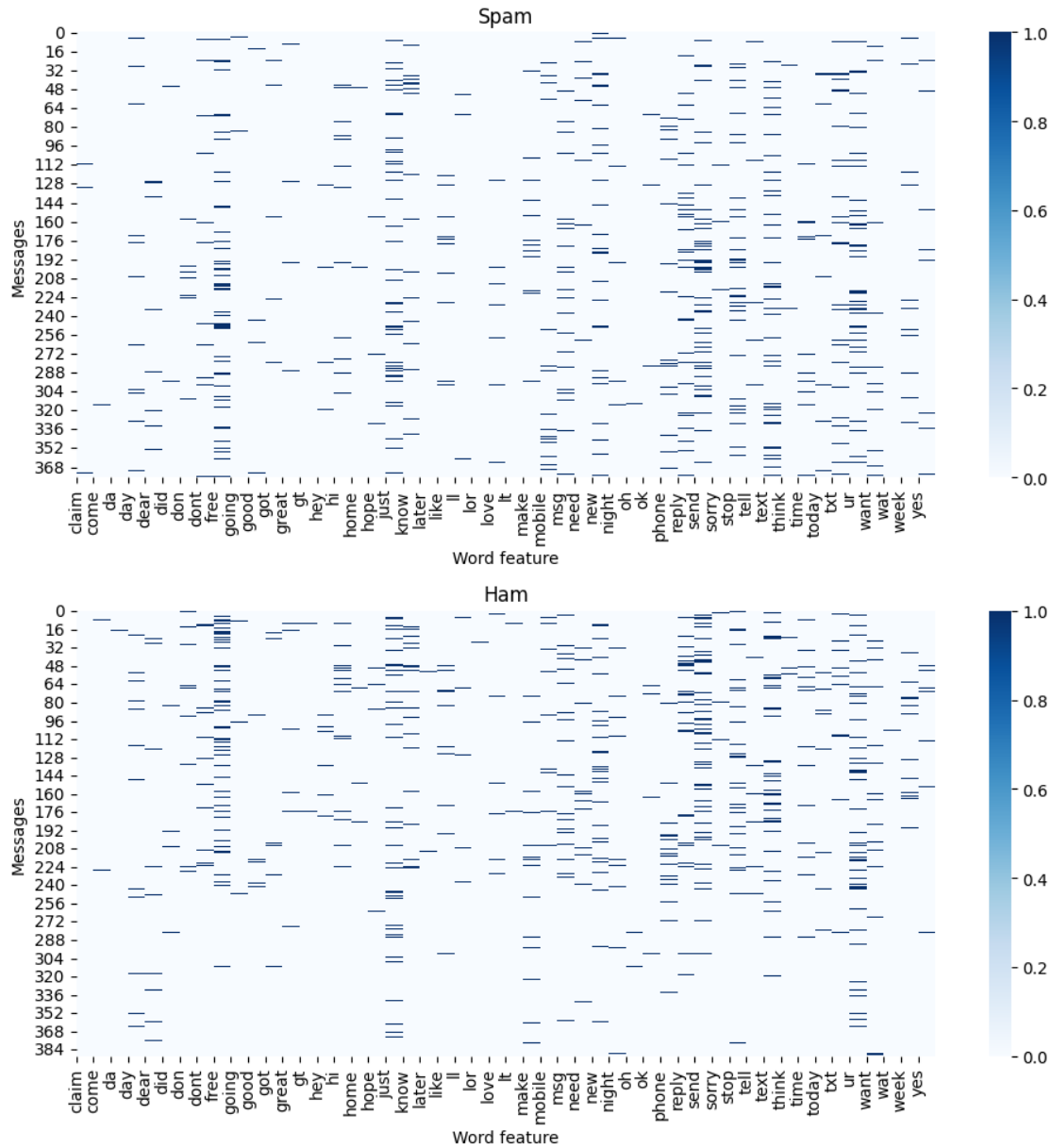
```
[33]: # Draw a heatmap of the support vectors in each class
sv = svm_lin[-1].support_vectors_.toarray() # support vectors
sv_idx = svm_lin[-1].support_ # indices of support vectors
sv_labels = y_train[sv_idx] # labels of support vectors
sv_spam = pd.DataFrame(
    sv[sv_labels == 1,:],
    columns=feature_names)
```

```

)
sv_ham = pd.DataFrame(
    sv[sv_labels == 0,:],
    columns=feature_names
)

fig, ax = plt.subplots(2,1,figsize=(10,10))
sns.heatmap(sv_spam, cmap='Blues', vmin=0, vmax=1, ax=ax[0])
ax[0].set_xlabel('Word feature')
ax[0].set_ylabel('Messages')
ax[0].set_title('Spam')
ax[0].set_xticks(np.arange(len(feature_names)), labels = feature_names)
sns.heatmap(sv_ham, cmap='Blues', vmin=0, vmax=1, ax=ax[1])
ax[1].set_xlabel('Word feature')
ax[1].set_ylabel('Messages')
ax[1].set_title('Ham')
ax[1].set_xticks(np.arange(len(feature_names)), labels = feature_names)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```



Q10b)

- Words with positive coefficients (e.g. “claim”, “mobile”, “txt”, “free”) will push the model to predict the message as spam, while words with negative coefficients (e.g. “later”, “it”, “come”, “hope”) will push the model to predict the message as not spam (thus, ham).
- Therefore, the support vector heatmaps show that spam support vectors will frequently contain these high-coefficient words, while ham support vectors are characterised by their absence.

### 4.0.3 Exercise 11 (CORE)

Now, let's explore using a nonlinear decision boundary: - Alter your pipeline from the previous exercise to use a polynomial kernel of degree 2 with a coefficient of 1. - Tune the parameter C. - Display the confusion matrix on the test data for both models. How do the results compare?

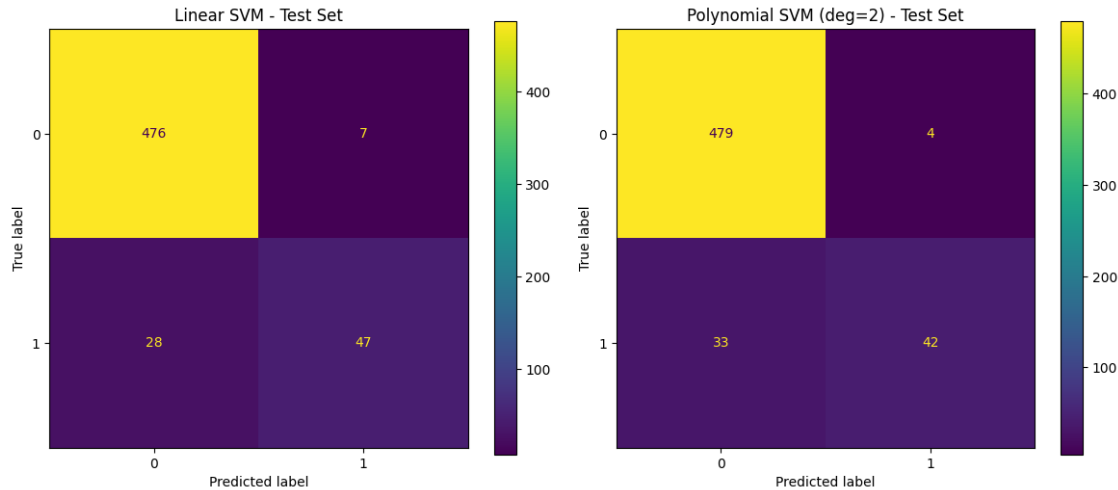
```
[34]: svm_poly_spam = make_pipeline(
        CountVectorizer(max_features = 50, stop_words = "english", binary = True),
        SVC(kernel = "poly", degree = 2, coef0 = 1)
    )

    # Tune C
    C = np.linspace(0.1, 10, 20)
    cv_poly_spam = GridSearchCV(
        svm_poly_spam,
        param_grid = {"svc__C": C},
        cv = StratifiedKFold(5, shuffle = True, random_state = 0),
        scoring = "accuracy"
    )
    cv_poly_spam.fit(X_train, y_train)
    print("Best C (poly spam):", cv_poly_spam.best_params_)
    print("Best CV accuracy (poly spam):", cv_poly_spam.best_score_)

    # Confusion matrices on test data
    fig, axes = plt.subplots(1, 2, figsize = (12, 5))
    ConfusionMatrixDisplay.from_estimator(cv_lin.best_estimator_, X_test, y_test,
        ↪ax=axes[0])
    axes[0].set_title("Linear SVM - Test Set")
    ConfusionMatrixDisplay.from_estimator(cv_poly_spam.best_estimator_, X_test,
        ↪y_test, ax=axes[1])
    axes[1].set_title("Polynomial SVM (deg=2) - Test Set")
    plt.tight_layout()
    plt.show()
```

Best C (poly spam): {'svc\_\_C': 0.1}

Best CV accuracy (poly spam): 0.9379748976623026



Q11)

Both models achieve high accuracy on the test set. The polynomial kernel (degree=2) captures interaction terms between words, which can help detect spam patterns that require the co-occurrence of two words. However, the improvement over the linear SVM is not significant given that there are only 50 bag-of-words features.

#### 4.0.4 Exercise 12 (CORE)

Try increasing the maximum number of features extracted in the bag of words for the linear SVM.

- Visualize the differences between the support vectors for the two classes. Are there any changes to the words which are relevant to distinguishing between the classes?
- How do the results compare on the test data?

```
[35]: svm_lin_more = make_pipeline(
    CountVectorizer(max_features = 200, stop_words = "english", binary = True),
    SVC(kernel = "linear", C = 1.0)
)
svm_lin_more.fit(X_train, y_train)

# Confusion matrix
ConfusionMatrixDisplay.from_estimator(svm_lin_more, X_test, y_test)
plt.title("Linear SVM (200 features) - Test Set")
plt.show()

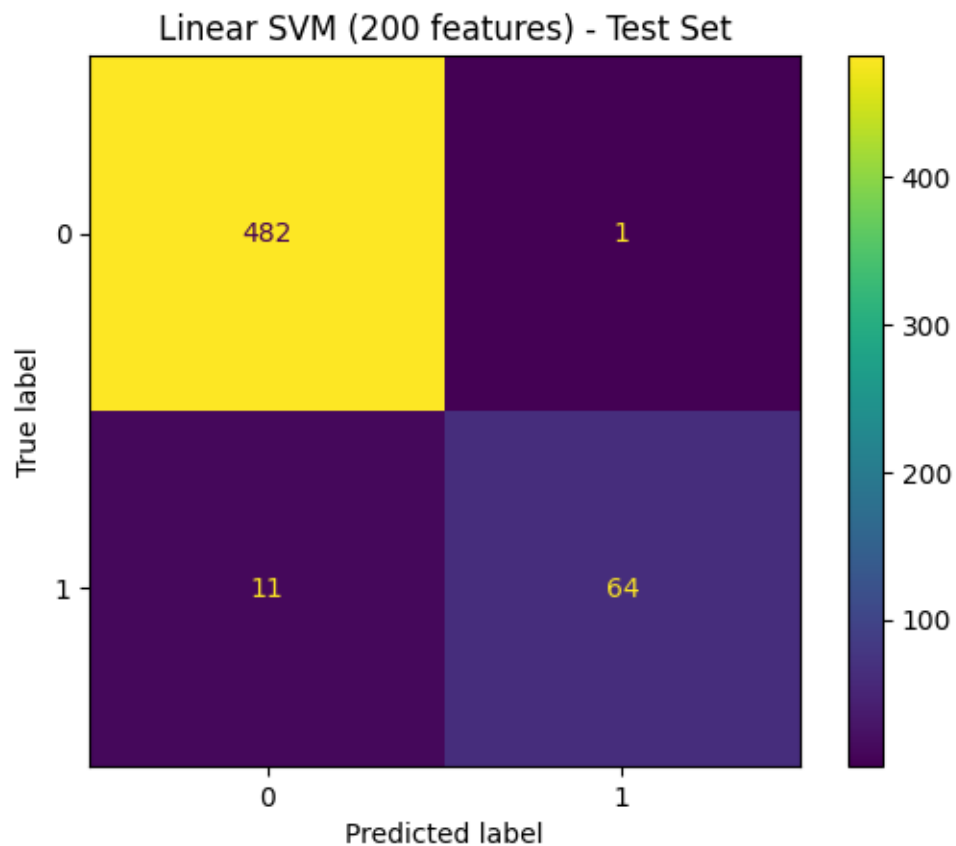
# Support vector heatmap
feature_names_200 = svm_lin_more[-1].get_feature_names_out()
sv_200 = svm_lin_more[-1].support_vectors_.toarray()
sv_idx_200 = svm_lin_more[-1].support_
sv_labels_200 = y_train[sv_idx_200]
```

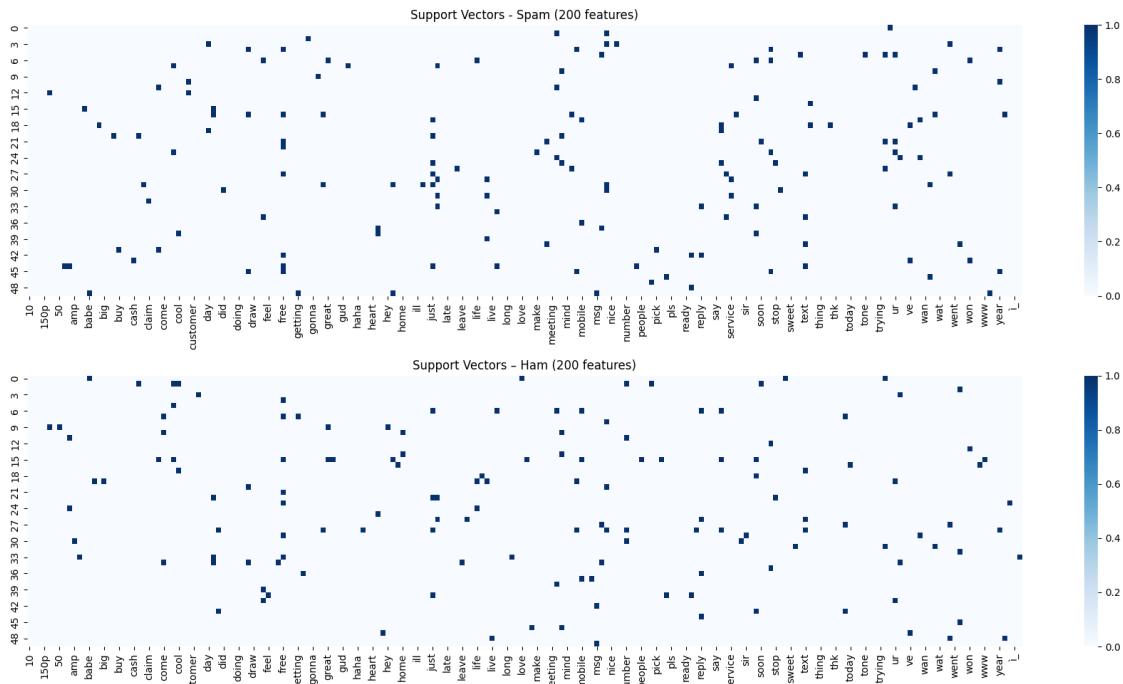
```

sv_spam_200 = pd.DataFrame(sv_200[sv_labels_200 == 1, :],
    ↪columns=feature_names_200)
sv_ham_200 = pd.DataFrame(sv_200[sv_labels_200 == 0, :],
    ↪columns=feature_names_200)

fig, ax = plt.subplots(2, 1, figsize = (18, 10))
sns.heatmap(sv_spam_200.iloc[:50, :], cmap = "Blues", vmin = 0, vmax = 1, ax =
    ↪ax[0])
ax[0].set_title("Support Vectors - Spam (200 features)")
sns.heatmap(sv_ham_200.iloc[:50, :], cmap = "Blues", vmin = 0, vmax = 1, ax =
    ↪ax[1])
ax[1].set_title("Support Vectors - Ham (200 features)")
plt.tight_layout()
plt.show()

```





Q12)

With 200 features, the model has access to a larger vocabulary which captures more discriminative spam signals. Hence, the test confusion matrix shows fewer misclassifications compared to the 50-feature model. The support vector heatmap now shows more words per message and reveals additional spam markers that were outside the top-50 vocabulary. Ham support vectors continue to be characterised by the absence of these spam keywords and the presence of ham keywords.

#### 4.0.5 Exercise 13 (EXTRA)

Lastly, try tuning also the number of features in your CV grid search, with a search over values of `C = np.array([0.1,1,5,10])` and `max_features = np.arange(50,500,10)`. - By comparing the different metrics, which value of `C` and `max_features` would you choose? - Fit the model with your selected values, and display the confusion matrix for the test data. How do the results compare to the previous models?

```
[36]: C = np.array([0.1,1,5,10])
max_features = np.arange(50,500,10)

svm_tune = make_pipeline(
    CountVectorizer(stop_words = "english", binary = True),
    SVC(kernel = "linear")
)

cv_tune = GridSearchCV(
    svm_tune,
```

```

param_grid={
    "countvectorizer__max_features": max_features,
    "svc__C": C_vals
},
cv = StratifiedKFold(5, shuffle = True, random_state = 0),
scoring=["accuracy", "f1", "recall", "precision"],
refit="f1"
)
cv_tune.fit(X_train, y_train)

print("Best params:", cv_tune.best_params_)
print("Best CV F1:", cv_tune.best_score_)

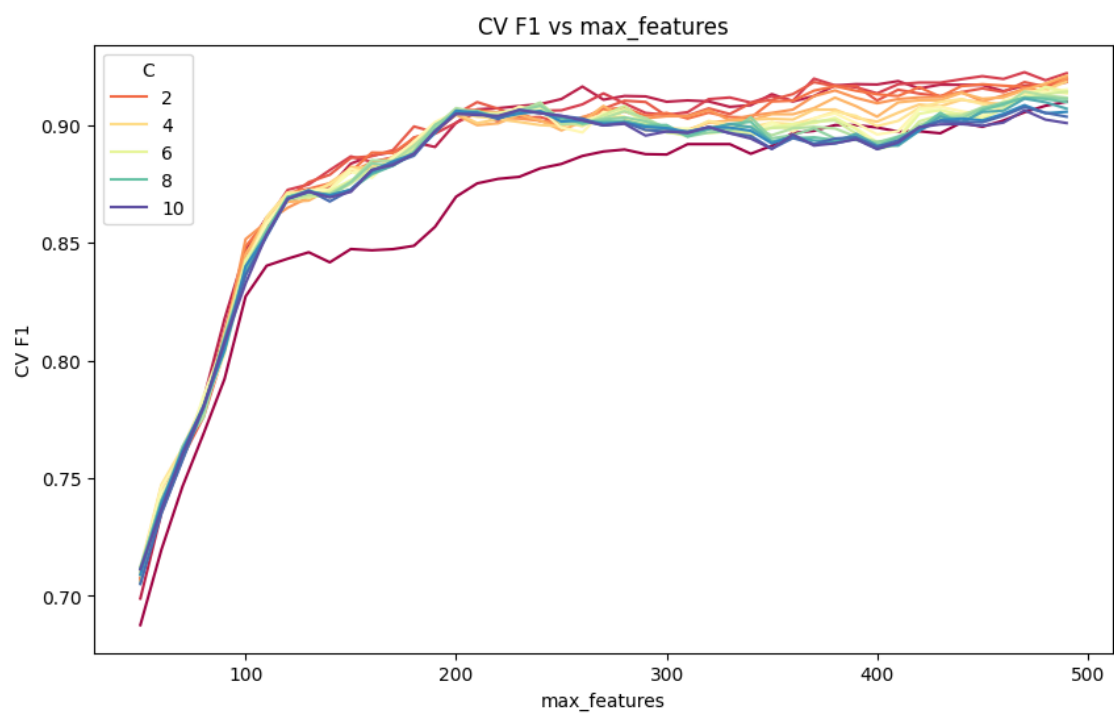
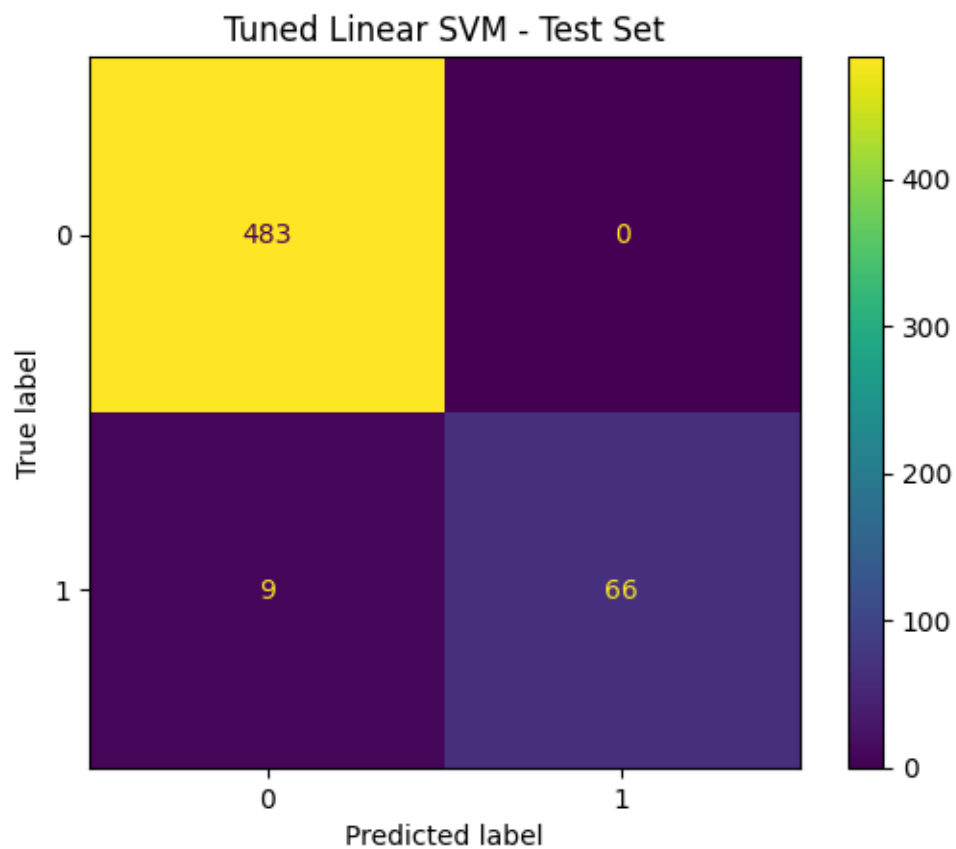
# Confusion matrix on test
ConfusionMatrixDisplay.from_estimator(cv_tune.best_estimator_, X_test, y_test)
plt.title("Tuned Linear SVM - Test Set")
plt.show()

# Plot F1 vs max_features for each C
cv_df = pd.DataFrame(cv_tune.cv_results_)
    ["param_countvectorizer__max_features", "param_svc__C", "mean_test_f1"]
].rename(columns = {
    "param_countvectorizer__max_features": "max_features",
    "param_svc__C": "C",
    "mean_test_f1": "CV F1"
})
plt.figure(figsize = (10, 6))
sns.lineplot(x = "max_features", y = "CV F1", hue = "C", data = cv_df, palette_
    ↪= "Spectral")
plt.title("CV F1 vs max_features")
plt.show()

```

Best params: {'countvectorizer\_\_max\_features': 470, 'svc\_\_C': 1.142105263157895}  
 Best CV F1: 0.9224409672204272





Q13)

- The joint grid search shows that F1 generally increases as max\_features increases, but plateaus over after 200.
- A small-moderate C (1-6) generally outperforms higher values of C (>8-10).
- Hence, I would choose C=1 and max\_features=200 for the simplest and optimal model, expected to have the least false negatives and positives.
- Based on the test confusion matrix with the model with the chosen best parameters, the model with the highest features have the least false negatives and no false positives, which means the highest recall and 100% precision, compared to the other models with fewer features.

## 5 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

```
[37]: # !jupyter nbconvert --to pdf mlp_week07.ipynb
```